Midterm

May 13, 2026

1 CV Midterm

Fasoli Marco - 5611520

1.1 Objective

In this assignment, we will work on change detection and blob matching tasks. Change detection is based on a loop, where at each time frame, you will compare the current frame I_t with a reference image or background B. Then blobs are computed and then we can compute the blob matching across adiacent frames using different descriptors.

1.2 Pipeline

1. **Background initialization:** the background is usually initialised (B_0) by taking an average of the first few frames (hoping/assuming they are empty).
2. **Change detection:** at each time t
   - Update background B_t (this is an optional step, in the first versions, you may set it constant to B_0)
   - Compute the change detection map M_t by comparing B_t with I_t
3. **Map cleaning:**
   - Cancel small bit regions
   - Apply morphological operations to make the change detection map smoother and more compact.
4. **Compute connected components** (also called BLOBS):
   - Each group of connected pixels marked as changes might be labelled with a common label, producing a connected component
   - To perform this step, we will use library functions (either skimage or opencv)
5. **Blob matching:**
   - At this point, the frame-based analysis is finished, and we can start discussing how we can connect components over time
   - To do so, we will compute matches between connected components in M_t and M_{t+1} matches may be based on vicinity (e.g., Euclidean distance between blob centroids) or even similarity (e.g., average intensity/colour within the blob, area, shape descriptors, ...)

1.3 Assumptions

All the operations that we are going to compute on image sequences will assume that: 1. Illumination does not vary in the observation interval. 2. The camera is still.

1.4 Imports

[1]:
```
import os
import numpy as np
from scipy import signal, spatial
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
from skimage import draw as drw
from skimage import data, color, exposure, img_as_float, img_as_ubyte,␣
↪morphology, filters, measure, util
from skimage import io
import cv2
import subprocess
from IPython.display import Video

import skimage
%matplotlib inline

PATH1 = 'station_video_frames/'
PATH2 = 'room_video_frames/'
ES1 = 'es1/'
```

1.5 Import Frames

In this first section we will define two useful functions that we are going to use to deal with images. In fact we need the images to be grayscaled and casted as float. Moreover since we are working with videos, we need a function that given a folder path, it creates an array of images that represent the video that we want to analyze.

[2]:
```
def preprocessing(img):
    """Returns the greyscaled version of the image, casting as a float the␣
↪values."""
    return img_as_float(color.rgb2gray(img))

def import_images(path):
    """Returns an array of images that represent a video."""
    images = []
    for filename in sorted(os.listdir(path)):
        full_path = os.path.join(path, filename)
        if os.path.isfile(full_path) and not filename.startswith('.'):
            try:
                img = io.imread(full_path)
                if img is not None:
                    images.append(preprocessing(img))
            except Exception as e:
                print(f"Skipping {filename}: {e}")
```

```
    return images
```

1.6 Import Background

In this part we need to define a function that given some of the initial frames (the ones that we assume to be empty), it computes the average of them in order to get a reference image (or background).

[3]:
```
def background_init(frames):
    """Returns the average of initial frames."""
    return np.average(frames,axis=0)
```

1.7 Export videos

In this section we are going to define some functions that will be essential in order to have a good visualization of the results after computing the change detenction and blob matching algorithm.

Let's define the first two functions for this scope: 1. **get_id_color:** it returns a color given a specific ID of a blob. In this way we can track (or visualize) with different colors multiple blobs, meaning that we can analyze more clearly the results returned by our pipeline. 2. **compute_visual_frame:** this function takes as input the frame and some information about the current blobs. In particular this function creates a yellow-ish mask to highlight the blobs, draws single points in order to track the movements and appends an ID to each of the blob in the frame.

[4]:
```
def get_id_color(obj_id):
    """Returns a specific, high-visibility BGR color based on the ID."""
    palette = [
        (0, 255, 0),       # Green
        (0, 165, 255),     # Orange
        (0, 0, 255),       # Red
        (255, 255, 0),     # Cyan
        (255, 0, 255),     # Magenta
        (0, 255, 255),     # Yellow
        (255, 191, 0),     # Deep Sky Blue
        (180, 105, 255)    # Hot Pink
    ]
    return palette[int(obj_id) % len(palette)]

def compute_visual_frame(frame, label_image, props, blob_to_id, track_history):
    """Returns an updated frame with per-ID blob colors and short motion links.
 ↪"""
    temp_frame = frame.copy()
    if temp_frame.dtype.kind == 'f':  # If it's a float (0.0-1.0)
        if temp_frame.max() <= 1.01:
            temp_frame = (temp_frame * 255).astype(np.uint8)
        else:
            temp_frame = temp_frame.astype(np.uint8)
    else:
```

```
        temp_frame = temp_frame.astype(np.uint8)

    if len(temp_frame.shape) == 2:
        visual_frame = cv2.cvtColor(temp_frame, cv2.COLOR_GRAY2RGB)
    else:
        visual_frame = temp_frame.copy()

    # Per-ID colored blob overlay.
    mask_layer = visual_frame.copy()
    for i, prop in enumerate(props):
        tid = blob_to_id.get(i)
        if tid is None:
            continue
        mask_layer[label_image == prop.label] = get_id_color(tid)
    visual_frame = cv2.addWeighted(mask_layer, 0.35, visual_frame, 0.65, 0)

    # Draw only current-frame motion segments.
    current_ids = set(blob_to_id.values())
    for tid in current_ids:
        path = track_history.get(tid, [])
        if len(path) >= 2:
            cv2.line(visual_frame, path[-2], path[-1], get_id_color(tid), 2)

    # Draw ID labels (and bbox) with consistent ID color.
    for i, prop in enumerate(props):
        tid = blob_to_id.get(i)
        if tid is None:
            continue
        color = get_id_color(tid)
        minr, minc, maxr, maxc = prop.bbox
        cv2.rectangle(visual_frame, (minc, minr), (maxc, maxr), color, 1)
        cv2.putText(visual_frame, f"ID:{tid}", (minc, max(minr - 5, 8)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.4, color, 1)

    return visual_frame

def plot_binary_maps(_frame, FRAME_IDX, diff, mask_tau, TAU, mask_3s, tau_3s,␣
 ↪K_SIGMA, mask_mo, tau_mo, mask_ot, tau_ot):
    """Plots the binary maps using the different threshold techniques."""
    fig, axes = plt.subplots(2, 3, figsize=(14, 7))
    axes[0, 0].imshow(_frame, cmap="gray")
    axes[0, 0].set_title(f"Frame {FRAME_IDX}")
    axes[0, 0].axis("off")

    axes[0, 1].imshow(diff, cmap="gray", vmin=0, vmax=1)
    axes[0, 1].set_title(r"$D = |I - B|$")
    axes[0, 1].axis("off")
```

```
    axes[0, 2].imshow(mask_tau, cmap="gray")
    axes[0, 2].set_title(fr"Fixed theshold ($\tau$ = {TAU:.2f})")
    axes[0, 2].axis("off")

    axes[1, 0].imshow(mask_3s, cmap="gray")
    axes[1, 0].set_title(fr"{K_SIGMA}$\sigma$ ($\tau$ = {tau_3s:.2f})")
    axes[1, 0].axis("off")

    axes[1, 1].imshow(mask_mo, cmap="gray")
    axes[1, 1].set_title(fr"mean(D) + offset ($\tau$ = {tau_mo:.2f})")
    axes[1, 1].axis("off")

    axes[1, 2].imshow(mask_ot, cmap="gray")
    axes[1, 2].set_title(fr"Otsu on D ($\tau$ = {tau_ot:.2f})")
    axes[1, 2].axis("off")

    plt.suptitle("Binary change maps for the four  heuristics", y=1.02)
    plt.tight_layout()
    plt.show()

def side_by_side_video(frames1, frames2, text1, text2):
    """Displays two videos one next to the other, supporting color and binary␣
 ↪maps."""
    n = min(len(frames1), len(frames2))
    side_by_side_frames = []

    for i in range(n):
        left = frames1[i]
        right = frames2[i]

        # 1. Handle Data Types: Convert float [0,1] or binary to uint8 [0,255]
        def to_uint8(img):
            if img.dtype.kind == 'f' or img.max() <= 1:
                return (np.clip(img, 0, 1) * 255).astype(np.uint8)
            return np.clip(img, 0, 255).astype(np.uint8)

        left_u8 = to_uint8(left)
        right_u8 = to_uint8(right)

        # 2. Ensure 3 channels (BGR) so concatenation and color text work
        def ensure_bgr(img):
            if len(img.shape) == 2:  # Binary or Grayscale
                return cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
            return img

        left_rgb = ensure_bgr(left_u8)
```

```
        right_rgb = ensure_bgr(right_u8)

        # 3. Concatenate horizontally
        canvas = np.concatenate([left_rgb, right_rgb], axis=1)
        h, w, _ = canvas.shape
        sep_x = w // 2

        # Visual separator
        canvas[:, sep_x-1:sep_x+1] = 255

        # Labels
        cv2.putText(canvas, text1, (10, 24),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2, cv2.
↪LINE_AA)
        cv2.putText(canvas, text2, (sep_x + 10, 24),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2, cv2.
↪LINE_AA)

        side_by_side_frames.append(canvas)

    return side_by_side_frames
```

We are going to work on videos, meaning that we need also some functions that given a list of images they export them into a video that can be reproduced in the notebook environment. In particular the following functions achieve precisely this goal:

[5]:
```
def export_video(frames, output_filename):
    """Export a video given a list of images, ensuring data is actually written.
↪"""
    if not frames:
        print("No frames provided for export.")
        return

    height = frames[0].shape[0]
    width = frames[0].shape[1]
    size = (width, height)
    fps = 24

    # Initialize the VideoWriter
    out = cv2.VideoWriter(output_filename, cv2.VideoWriter_fourcc(*'mp4v'),␣
↪fps, size)

    for frame in frames:
        # 1. Handle Data Types: Convert float [0,1] to uint8 [0,255]
        if frame.dtype.kind == 'f':
            img_to_write = (np.clip(frame, 0, 1) * 255).astype(np.uint8)
        else:
```

```
            img_to_write = frame.astype(np.uint8)

        # 2. Handle Color: Ensure BGR format for OpenCV
        if len(img_to_write.shape) == 2:
            img_to_write = cv2.cvtColor(img_to_write, cv2.COLOR_GRAY2BGR)
        elif img_to_write.shape[2] == 3:
            img_to_write = cv2.cvtColor(img_to_write, cv2.COLOR_RGB2BGR)

        out.write(img_to_write)

    # Release the writer to finalize the file
    out.release()
    print(f"Video saved as {output_filename}")

def show_video(input_file):
    """Show in the notebook environment a given video."""
    output_file = 'video.mp4'
    cmd = f"ffmpeg -i {input_file} -vcodec libopenh264 -f mp4 -y {output_file}"
    try:
        subprocess.run(cmd.split(), check=True, capture_output=True, text=True)
        print("Conversion successful! Displaying video...")
        display(Video(output_file, embed=True, width=800))
    except subprocess.CalledProcessError as e:
        print("FFMPEG Error:")
        print(e.stderr)

def export_and_show_video(frames, output_filename):
    """Executes the previous functions."""
    export_video(frames, output_filename)
    show_video(output_filename)
```

1.8 Change detection and thresholding functions

Since we are working on image sequences gathered from a still camera, the operation of motion segemntation is called change detection. The latter term refers to the operation of detecting what has changed between the current frame I_t and a reference image (or background) I_{ref}. These changes can be computed by the following operation:

$$M_t(x, y) = \begin{cases} 1 & \text{if} \quad |I_t(x, y) - I_{ref}(x, y)| > \tau \\ 0 & \text{otherwise} \end{cases}$$

where τ is a threshold that can be fixed or computed with heuristics or algorithms and $M_t(x, y)$ is a binary map that highlights the points that have been changed significantly. Its value depends on the acquisition device and acquisition conditions.

Let's first define the functions that we will be using in order to compute $M_t(x, y)$ given a frame and the reference image:

[6]:
```python
def difference_map(frame, ref):
    """Absolute difference |I - B|."""
    f = np.asarray(frame, dtype=np.float64)
    r = np.asarray(ref, dtype=np.float64)
    return np.abs(f - r)

def binarize_change(diff, tau):
    """Binary change mask: D > tau."""
    return diff > tau
```

1.8.1 How to choose τ

As we said before, the binary map computed by the change detection is heavily influenced by the value of τ, that can be computed in different ways. In this project we are going to analyze the effect of τ using different strategies:

- **Fixed / manual:** chosen by trial and error.
- $k\sigma$ **rule:** this heuristic is based on the assumption that the noise in the difference map follows a (quasi) Gaussian distribution.
  - By calculating the standard deviation (σ) of the difference image $|I_t - I_{ref}|$, we set $\tau = k\sigma$.
  - Using $k = 3$ is conservative (fewer false positives), while lower values such as $k = 1$ or $k = 2$ increase sensitivity but may include more noise.
- **Mean + offset:** this is a dynamic thresholding strategy where τ is computed as the average intensity of the difference map plus an offset.
  - The formula is $\tau = \text{mean}(|I_t - I_{ref}|) + \text{offset}$.
  - The mean accounts for global changes in the scene's brightness, while the offset allows for fine-tuning the sensitivity. A higher offset reduces "ghosting" and false positives but might cause you to lose smaller or slower-moving objects.
- **Otsu's method:** Otsu selects a threshold by maximizing between-class variance of the **histogram** of diff. It works best when the histogram is approximately **bimodal** (e.g. unchanged vs changed pixels).

At this point let's define the functions that compute the value of τ:

[7]:
```python
def tau_ksigma(diffs, k=3.0):
    """Returns tau = k*sigma for sigma-based thresholding."""
    sigma = np.std(diffs)
    tau = k * sigma
    return tau if tau >= 0.07 else 0.07

def tau_mean_offset(diff, offset=20):
    """Returns the tau using the mean + offset heuristics."""
    avg = np.mean(diff)
    return avg + offset

def tau_otsu(diff):
    """Returns the tau computed by using Otsu algorithm on diff."""
```

```
    d = np.asarray(diff)
    if np.ptp(d) == 0:
        return float(d.ravel()[0])
    tau = float(filters.threshold_otsu(d))
    return tau if tau >= 0.07 else 0.07
```

Note that for certain tau_* functions we fixed a lower bound for the possible threshold. This will be useful when we will work on the second video, since if the two adiacent frames are almost equal the resulting threshold could be too low.

1.8.2 Change detection and running average

Up to this point, we always referred to the reference image as it was fixed. That could be a case, but we can also update the reference image at each iteration so as to deal with slow changing background. This method is called *running average* and it's quite robust to moving object in the scene and it incorporates smooth stable changes. In order to update the reference image, we are going to compute the same steps as before but we add the following formula:

$$I_{ref}(x,y) = B_t(x,y) = \begin{cases} B_{t-1}(x,y) & \text{if} \quad |I_t(x,y) - B_{t-1}(x,y)| > \tau \\ (1-\alpha) * B_{t-1} + \alpha * I_t(x,y) & \text{otherwise} \end{cases}$$

where α is a fixed value that help to incorporates smooth changes.

For this reason, in this project we are going to define the functions that uses a fixed reference image (change_detection_*) but also functions with changing background (running_average_*):

[8]:
```
def change_detection_with_tau(frame, ref, tau):
    """Binarize using an explicit tau. (Fixed background)"""
    diff = difference_map(frame, ref)
    return binarize_change(diff, tau), diff

def change_detection_ksigma(frame, ref, k=3.0):
    """Binarize using ksigma heuristic. (Fixed background)"""
    diff = difference_map(frame, ref)
    tau = tau_ksigma(diff, k=k)
    return binarize_change(diff, tau), diff, tau

def change_detection_mean(frame, ref, offset):
    """Binarize using mean + offset heuristic. (Fixed background)"""
    diff = difference_map(frame, ref)
    tau = tau_mean_offset(diff, offset)
    return binarize_change(diff, tau), diff, tau

def change_detection_otsu(frame, ref):
    """Binarize using Otsu algorithm. (Fixed background)"""
    diff = difference_map(frame, ref)
    tau = tau_otsu(diff)
    return binarize_change(diff, tau), diff, tau
```

[9]:
```
def running_average_with_tau(frame, ref, tau, alpha=0.1):
    """Binarize using an explicit tau. (Updates background)"""
    changes, _ = change_detection_with_tau(frame, ref, tau)
    return changes, np.where(changes, ref, (1 - alpha) * ref + alpha * frame)

def running_average_ksigma(frame, ref, k=3.0, alpha=0.1):
    """Binarize using ksigma heuristic. (Updates background)"""
    changes, _, _ = change_detection_ksigma(frame, ref, k=k)
    return changes, np.where(changes, ref, (1 - alpha) * ref + alpha * frame)


def running_average_mean(frame, ref, offset, alpha=0.1):
    """Binarize using mean + offset heuristic. (Updates background)"""
    changes, _, _ = change_detection_mean(frame, ref, offset)
    return changes, np.where(changes, ref, (1 - alpha) * ref + alpha * frame)


def running_average_otsu(frame, ref, alpha=0.1):
    """Binarize using Otsu algorithm. (Updates background)"""
    changes, _, _ = change_detection_otsu(frame, ref)
    return changes, np.where(changes, ref, (1 - alpha) * ref + alpha * frame)

def running_average_with_tau_temporal(frame, ref, frame_t_minus_delta, tau,␣
↪tau_prime=None, alpha=0.1):
    """
    Change detection with explicit tau and temporally-gated background update.

    Update rule:
      B_t(x,y) = B_{t-1}(x,y)                                  if␣
↪|I_t(x,y)-I_{t-delta}(x,y)| > tau'
                 (1-alpha)B_{t-1}(x,y) + alpha I_t(x,y) otherwise
    """
    if tau_prime is None:
        tau_prime = tau

    changes, _ = change_detection_with_tau(frame, ref, tau)
    temporal_diff = difference_map(frame, frame_t_minus_delta)
    freeze_bg = temporal_diff > tau_prime
    updated_bg = np.where(freeze_bg, ref, (1 - alpha) * ref + alpha * frame)
    return changes, updated_bg
```

Note that we added the running_average_with_tau_temporal that also deal with not slow changing background, and we are going to see it's application in the second video. The formula for updating the background is the following:

$$I_{ref}(x,y) = B_t(x,y) = \begin{cases} B_{t-1}(x,y) & \text{if} \quad |I_t(x,y) - I_{t-\Delta}(x,y)| > \tau' \\ (1-\alpha) * B_{t-1} + \alpha * I_t(x,y) & \text{otherwise} \end{cases}$$

We also need some functions that will be useful to tune some of the hyper-parameters:

[10]:
```
def plot_tune_k_sigma(frame, ref, ks=(1.0, 2.0, 3.0), figsize=None):
    """Plot one frame and binary masks for three k·  thresholds (same rule as␣
↪change_detection_ksigma)."""
    if figsize is None:
        figsize = (4 * (len(ks) + 1), 4)
    fig, axes = plt.subplots(1, len(ks) + 1, figsize=figsize)
    axes[0].imshow(frame, cmap='gray')
    axes[0].set_title('Frame')
    for ax, k in zip(axes[1:], ks):
        mask, _, tau = change_detection_ksigma(frame, ref, k=k)
        ax.imshow(mask, cmap='gray')
        ax.set_title(f'k={k:g},  ={tau:.4f}')
    for ax in axes:
        ax.axis('off')
    plt.tight_layout()
    return fig, axes


def plot_tune_mean_offset(frame, ref, offsets=(0.05, 0.1, 0.2), figsize=None):
    """Plot one frame and binary masks for three mean(|D|)+offset thresholds."""
    if figsize is None:
        figsize = (4 * (len(offsets) + 1), 4)
    fig, axes = plt.subplots(1, len(offsets) + 1, figsize=figsize)
    axes[0].imshow(frame, cmap='gray')
    axes[0].set_title('Frame')
    for ax, off in zip(axes[1:], offsets):
        mask, _, tau = change_detection_mean(frame, ref, offset=off)
        ax.imshow(mask, cmap='gray')
        ax.set_title(f'offset={off:g},  ={tau:.4f}')
    for ax in axes:
        ax.axis('off')
    plt.tight_layout()
    return fig, axes
```

Lastly, we need some functions that will be useful when we are going to compare the algorithm on the complete sequence:

[11]:
```
def collect_taus_sequence(frames, initial_bg, method, tau=0.1, offset=0.1, k=3.
↪0, alpha=0.1):
    """Per-frame  for each detection method (same dynamics as␣
↪compute_blobs_with_tracking, pass 1)."""
    taus = []
    bg = initial_bg.copy()
    for frame in frames:
        if method == 'manual_static':
            taus.append(float(tau))
```

```
        elif method == 'ksigma_static':
            taus.append(float(tau_ksigma(difference_map(frame, bg), k=k)))
        elif method == 'mean_static':
            taus.append(float(tau_mean_offset(difference_map(frame, bg),␣
↪offset)))
        elif method == 'otsu_static':
            taus.append(float(tau_otsu(difference_map(frame, bg))))
        elif method == 'manual_adaptive':
            taus.append(float(tau))
            _, bg = running_average_with_tau(frame, bg, tau, alpha)
        elif method == 'ksigma_adaptive':
            _, _, t = change_detection_ksigma(frame, bg, k=k)
            taus.append(float(t))
            _, bg = running_average_ksigma(frame, bg, k=k, alpha=alpha)
        elif method == 'mean_adaptive':
            _, _, t = change_detection_mean(frame, bg, offset)
            taus.append(float(t))
            _, bg = running_average_mean(frame, bg, offset, alpha)
        elif method == 'otsu_adaptive':
            _, _, t = change_detection_otsu(frame, bg)
            taus.append(float(t))
            _, bg = running_average_otsu(frame, bg, alpha)
        else:
            raise ValueError(f"Unknown method '{method}'")
    return np.asarray(taus, dtype=np.float64)


def summarize_tau_sequence(taus):
    """Mean, min, and max  over a sequence."""
    taus = np.asarray(taus, dtype=np.float64)
    if taus.size == 0:
        return {'mean': np.nan, 'min': np.nan, 'max': np.nan, 'n': 0}
    return {
        'mean': float(np.mean(taus)),
        'min': float(np.min(taus)),
        'max': float(np.max(taus)),
        'n': int(taus.size),
    }
```

1.9 Map cleaning

In this section we are going to define a function that uses a morphological procedure that removes small objects. In this way we obtain a cleaned binary map that does not have tiny spikes all over the map, but instead have all the features that allow us to compute the connected components step:

[12]:
```
def map_cleaning(changes):
    """Returns the cleaned version of the binary map."""
    final_changes = morphology.closing(changes,morphology.disk(2))
    return morphology.remove_small_objects(final_changes,100)
```

In particular: - **morphology.closing:** performs a Dilation followed by an Erosion. In this way the blobs stay roughly the same size, but their internal holes are filled, and their outer boundaries are smoothed. Using a disk(2) (a circular shape with a radius of 2 pixels) means it will fill holes or gaps up to about 4-5 pixels wide. - Dilation: expands the white areas (the blobs). This effectively "swallows" small dark holes inside the blobs and connects parts of an object that are separated by just 1 or 2 pixels. - Erosion: shrinks the object back down to its original size. - **morphology.remove_small_objects:** this wipes away "salt-and-pepper" noise—tiny points that aren't large enough to be the "objects" we actually want to track.

1.10 Connected components

At this point we have to define a function that computed the connected components from the cleaned version of the binary map:

[13]:
```
def filter_small_area_blobs(label_image, min_area=100):
    """Remove connected components smaller than min_area and relabel."""
    props = measure.regionprops(label_image)
    keep_labels = [p.label for p in props if p.area >= min_area]

    if len(keep_labels) == 0:
        return np.zeros_like(label_image)

    filtered = np.isin(label_image, keep_labels)
    return measure.label(filtered)


def compute_cc(changes, min_area=100):
    """Returns connected components after small-area blob filtering."""
    label_image = measure.label(changes)
    label_image = filter_small_area_blobs(label_image, min_area=min_area)
    return label_image, measure.regionprops(label_image)
```

In particular: - **measure.label(changes):** this function looks at the binary mask and groups adjacent white pixels together (indentifying them with an ID). - **filter_small_area_blobs(label_image, min_area=100):** removes the blobs with a small area. - **measure.regionprops(label_image):** it returns a list of properties of the blobs such as: centroid, area, bbox, eccentricity and so on. These features will be exploited in the next step in order to compute the match between blobs of adiacent frames.

1.11 Blob matching

Once the individual connected components (blobs) have been identified in each frame, the next step is to link them across the temporal dimension. While change detection tells us which pixels have

changed, blob matching allows us to understand the movement of specific objects over time. The objective of this step is to compute a correspondence between the blobs detected in the previous frame and those detected in the current frame.

1.11.1 How to match Blobs

To decide which blob in the current frame is the most likely candidate for a blob from the previous frame, we compare several physical descriptors. A "cost" is calculated for each pair of blobs based on these features: - **Spatial Proximity (Centroid):** based on the assumption that objects move continuously, a blob in the current frame should be close to its position in the previous frame. - **Size Consistency (Area):** physical objects generally do not change their total size from one frame to the next. - **Intensity similarity:** the average brightness or color within a blob should remain relatively stable under constant lighting conditions. - **Shape similarity:** eccentricity helps to distinguish between different types of objects, such as a person versus a vehicle.

So let's define the functions that computes the vicinity / similarity between blobs:

[14]:
```
def pairwise_centroid_distance(blobs1, blobs2):
    """Returns Euclidean distances between blob centroids."""
    c1 = np.asarray([b.centroid for b in blobs1], dtype=np.float64)
    c2 = np.asarray([b.centroid for b in blobs2], dtype=np.float64)
    if c1.size == 0 or c2.size == 0:
        return np.empty((len(blobs1), len(blobs2)))
    return spatial.distance.cdist(c1,c2)

def blob_mean_intensity(frame, blob):
    """Mean grayscale on pixels belonging to the blob."""
    cr = blob.coords
    return float(np.mean(frame[cr[:, 0], cr[:, 1]]))

def pairwise_intensity_absdiff(blobs1, blobs2, frame1, frame2):
    """Returns the absolute difference between the intenisity of two blobs."""
    m1 = np.array([blob_mean_intensity(frame1, b) for b in blobs1], dtype=np.
 ↪float64)[:, None]
    m2 = np.array([blob_mean_intensity(frame2, b) for b in blobs2], dtype=np.
 ↪float64)[None, :]
    return np.abs(m1 - m2)

def pairwise_logarea_absdiff(blobs1, blobs2):
    """Returns the absolute difference between the log of the area of two blobs.
 ↪"""
    a1 = np.array([max(b.area, 1) for b in blobs1], dtype=np.float64)[:, None]
    a2 = np.array([max(b.area, 1) for b in blobs2], dtype=np.float64)[None, :]
    return np.abs(np.log(a1) - np.log(a2))

def pairwise_eccentricity_absdiff(blobs1, blobs2):
    """Returns the absolute difference between the eccentricity of two blobs."""
    e1 = np.array([b.eccentricity for b in blobs1], dtype=np.float64)[:, None]
```

```
    e2 = np.array([b.eccentricity for b in blobs2], dtype=np.float64)[None, :]
    return np.abs(e1 - e2)

def _normalize01(M):
    """Scale entries to [0, 1] to ensure all features have equal weight␣
 ↪influence."""
    m = np.asarray(M, dtype=np.float64)
    lo, hi = np.min(m), np.max(m)
    if hi <= lo + 1e-12:
        return np.zeros_like(m)
    return (m - lo) / (hi - lo)
```

Now that we have the vicinity / similarity between blobs, we need to define the blob matching procedure logic for connecting objects between frame I_t and I_{t+1}. It follows a Feature-Based Matching approach: 1. For every detected blob, we extract a feature vector including its centroid (x, y), mean intensity, area, and eccentricity. 2. We construct a "Super Cost" matrix C. For every pair of blobs (Blob i from the previous frame and Blob j from the current frame), we calculate a weighted cost:

$$C_{i,j} = w_{dist} * dist(i, j) + w_{int} * int(i, j) + w_{area} * area(i, j) + w_{ecc} * ecc(i, j)$$

where dist(i, j), int(i,j), area(i, j), ecc(i, j) is the absolute difference between that feature of the blob i and blob j, and w_* is the weight that we are going to assign to each descriptor. Note that all feature differences are normalized to a $[0, 1]$ range to ensure fair weighting. 3. We iterate through the blobs of the previous frame. For each blob, we look for the "cheapest" available match in the current frame. Once a blob in the current frame is "taken," it cannot be assigned to another object, ensuring a one-to-one mapping where possible. 4. If the lowest available cost exceeds a *max_cost* threshold, we assume the object has left the scene or was misidentified, and no match is made.

Note that this procedure will be applied to the complete image sequence, meaning that we will compute the blob matching across frame t and frame $t + 1$, for all t from 0 to the last frame.

[15]:
```
def match_blobs(blobs1, blobs2, frame1, frame2, w_dist=1.0, w_int=0, w_area=0,␣
 ↪w_ecc=0, max_cost=None):
    """
    Greedy one-to-one matching.
    """
    D = pairwise_centroid_distance(blobs1, blobs2)
    if D.size == 0:
        return np.empty((0, 2), dtype=int)

    I = pairwise_intensity_absdiff(blobs1, blobs2, frame1, frame2)
    A = pairwise_logarea_absdiff(blobs1, blobs2)
    E = pairwise_eccentricity_absdiff(blobs1, blobs2)

    C = (
        w_dist * _normalize01(D)
        + w_int * _normalize01(I)
        + w_area * _normalize01(A)
```

```
        + w_ecc * _normalize01(E)
    )

    S = 1.0 - _normalize01(C)
    argmaxS_h = np.argmax(S, axis=1)
    argmaxS_v = np.argmax(S, axis=0)

    match = []
    for i, amx in enumerate(argmaxS_h):
        if argmaxS_v[amx] != i:
            continue
        if max_cost is not None and C[i, amx] > max_cost:
            continue
        match.append((i, int(amx)))

    if len(match) == 0:
        return np.empty((0, 2), dtype=int)
    return np.array(match, dtype=int)
```

1.12 Pipeline

In this section, we are going to define a function that combines all the functions defined before, so as to get an image sequence in which each blob is highlighted and tracked. In order to do so, we also need to define a function that track blobs over time:

[16]:
```
def track_blobs(props, prev_props, prev_frame, frame, blob_to_id,␣
↪new_blob_to_id, next_id, track_history, **match_kwargs):
    """Track blobs over adiacent frames."""
    # Match previous blobs to current blobs.
    if prev_props is not None and len(props) > 0 and len(prev_props) > 0:
        _mk = {k: match_kwargs[k] for k in ('w_dist', 'w_int', 'w_area',␣
↪'w_ecc', 'max_cost') if k in match_kwargs}
        matches = match_blobs(prev_props, props, prev_frame, frame, **_mk)

        for m in matches:
            prev_idx, curr_idx = int(m[0]), int(m[1])

            # obj_id = id_map.get(idx1, next_id)
            obj_id = blob_to_id.get(prev_idx, next_id)
            if prev_idx not in blob_to_id:
                next_id += 1

            new_blob_to_id[curr_idx] = obj_id

            cy, cx = props[curr_idx].centroid
            if obj_id not in track_history:
                track_history[obj_id] = []
```

```
            track_history[obj_id].append((int(cx), int(cy)))

    # Assign IDs to unmatched current blobs.
    for i in range(len(props)):
        if i in new_blob_to_id:
            continue
        new_blob_to_id[i] = next_id
        cy, cx = props[i].centroid
        track_history[next_id] = [(int(cx), int(cy))]
        next_id += 1

    return track_history, new_blob_to_id, next_id

def compute_blobs_with_tracking(frames, initial_bg, method, sigma=0, tau=0,␣
 ↪offset=0, k=3.0, alpha=0, min_area=100, **match_kwargs):
    """
    Two-pass pipeline:
    1) compute blobs for all frames,
    2) propagate IDs frame-to-frame using track_blobs.
    """
    ALGO_REGISTRY = {
        "manual_static":  lambda f, b: (change_detection_with_tau(f, b,␣
 ↪tau)[0], b),
        "ksigma_static":  lambda f, b: (change_detection_ksigma(f, b, k=k)[0],␣
 ↪b),
        "mean_static":    lambda f, b: (change_detection_mean(f, b, offset)[0],␣
 ↪b),
        "otsu_static":    lambda f, b: (change_detection_otsu(f, b)[0], b),
        "manual_adaptive": lambda f, b: running_average_with_tau(f, b, tau,␣
 ↪alpha),
        "ksigma_adaptive": lambda f, b: running_average_ksigma(f, b, k=k,␣
 ↪alpha=alpha),
        "mean_adaptive":   lambda f, b: running_average_mean(f, b, offset,␣
 ↪alpha),
        "otsu_adaptive":   lambda f, b: running_average_otsu(f, b, alpha),
    }

    process_frame = ALGO_REGISTRY.get(method)
    if process_frame is None:
        raise ValueError(f"Method '{method}' not recognized. Choose from:␣
 ↪{list(ALGO_REGISTRY.keys())}")

    n = len(frames)
    if n == 0:
        return [], {}
```

```python
# Pass 1: detect blobs on all frames.
current_bg = initial_bg.copy()
labels_list, props_list = [], []
for frame in frames:
    mask, current_bg = process_frame(frame, current_bg)
    cleaned = map_cleaning(mask)
    label_img, props = compute_cc(cleaned, min_area=min_area)
    labels_list.append(label_img)
    props_list.append(props)

# Pass 2: plain ID propagation via track_blobs.
per_frame_blob_to_id = [dict() for _ in range(n)]
track_history = {}
next_id = 0

prev_props = None
prev_frame = None
blob_to_id = {}

for t in range(n):
    props = props_list[t]
    frame = frames[t]
    new_blob_to_id = {}

    track_history, new_blob_to_id, next_id = track_blobs(
        props,
        prev_props,
        prev_frame,
        frame,
        blob_to_id,
        new_blob_to_id,
        next_id,
        track_history,
        **match_kwargs,
    )

    per_frame_blob_to_id[t] = new_blob_to_id
    prev_props = props
    prev_frame = frame
    blob_to_id = new_blob_to_id

# Visualization pass.
processed_frames = []
history_so_far = {}
for t in range(n):
    for j, tid in per_frame_blob_to_id[t].items():
        cy, cx = props_list[t][j].centroid
```

```
            history_so_far.setdefault(tid, []).append((int(cx), int(cy)))
        processed_frames.append(
            compute_visual_frame(frames[t], labels_list[t], props_list[t],␣
↪per_frame_blob_to_id[t], history_so_far)
        )

    return processed_frames, track_history
```

As a last thing for blob tracking, we need some functions that allow us to compare the different results got by using different strategies to detect and track blobs:

[17]:
```
def summarize_tracks(track_history, min_len=5):
    """Returns the stats of a given track_history."""
    lengths = np.array([len(path) for path in track_history.values()], dtype=np.
↪int32)
    if lengths.size == 0:
        return {
            'n_tracks': 0,
            'mean_len': 0.0,
            'median_len': 0.0,
            'long_tracks(>=min_len)': 0,
            'short_ratio(<min_len)': 1.0,
        }

    return {
        'n_tracks': int(lengths.size),
        'mean_len': float(np.mean(lengths)),
        'median_len': float(np.median(lengths)),
        'long_tracks(>=min_len)': int(np.sum(lengths >= min_len)),
        'short_ratio(<min_len)': float(np.mean(lengths < min_len)),
    }

def print_stats(rows):
    """Prints the stats."""
    for name, s in rows:
        print(
            f"{name:15s} | "
            f"n_tracks={s['n_tracks']:3d} | "
            f"mean_len={s['mean_len']:.2f} | "
            f"median_len={s['median_len']:.2f} | "
            f"long_tracks={s['long_tracks(>=min_len)']:3d} | "
            f"short_ratio={s['short_ratio(<min_len)']:.3f}"
        )
```

1.13 First video

The first image sequence that we are going to analyze was already provided by the professors and it shows plenty of frames gathered from a surveillence camera in what it seems a train station. Here the two baseline assumptions hold, in fact the camera is still and the light intensity does not change over time. The objects that we are going to detect and match are people.

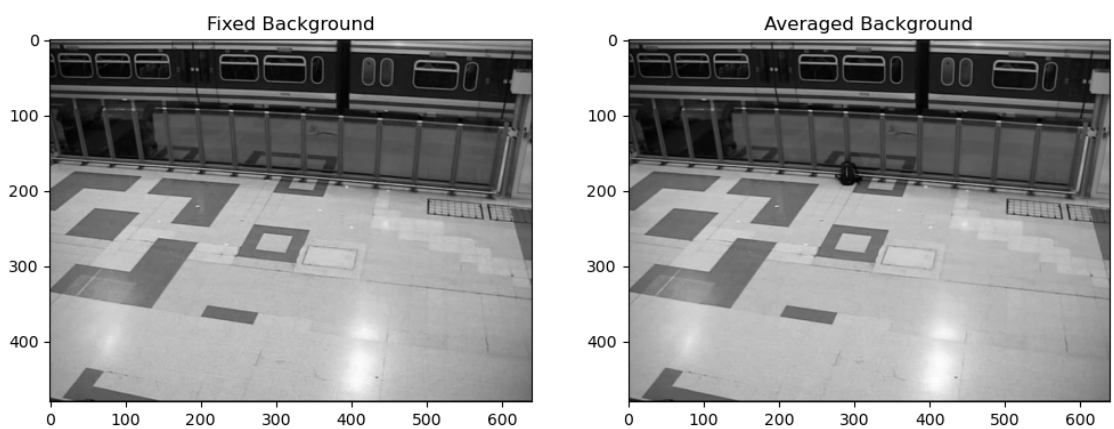
1.13.1 Background initialization

Let's first initialize the background, in the provided folder there are two ways to define the reference image. In fact there's a file named "Background.jpg" which represents the empty frame by itself but there are also two initial frames that we assume to be empty. For this reason we are going to import both of them so as to compare the results given a "ground truth" empty frame and the average of what we assume to be empty.

[18]:
```
bg_fixed = preprocessing(io.imread(PATH1 + ES1 + 'Background.jpg'))
bg_aux = []
bg_aux.append(preprocessing(io.imread(PATH1 + ES1 + 'EmptyScene01.jpg')))
bg_aux.append(preprocessing(io.imread(PATH1 + ES1 + 'EmptyScene02.jpg')))
diff_bg = difference_map(bg_aux[0], bg_aux[1])
bg_avg = background_init(bg_aux)
```

At this point, let's plot the reference images:

[19]:
```
plt.figure(figsize=(12,8))
plt.subplot(121)
plt.imshow(bg_fixed,cmap='gray')
plt.title('Fixed Background')
plt.subplot(122)
plt.imshow(bg_avg,cmap='gray')
plt.title('Averaged Background')
plt.show()
```



We can see that they are pretty similar, except the presence of a backpack in the averaged back-

ground. This could create different results during the pipeline, so we are going to analyze it with more detail in the next sections.

1.13.2 Change detection and different thresholds

At this point, we need to import the frames of the image sequence provided. Before diving into the full pipeline, we are going to analyze the effect of different threshold techinques on the change detection algorithm on a particular frame. So let's import the original video:

[20]:
```
es1_frames = import_images(PATH1 + ES1 + "video/")
```

Before diving into the analysis, let's visualize the frames that we are going to work on:

[21]:
```
export_and_show_video(es1_frames, 'original_es1.mp4')
```

```
Video saved as original_es1.mp4
Conversion successful! Displaying video…
```

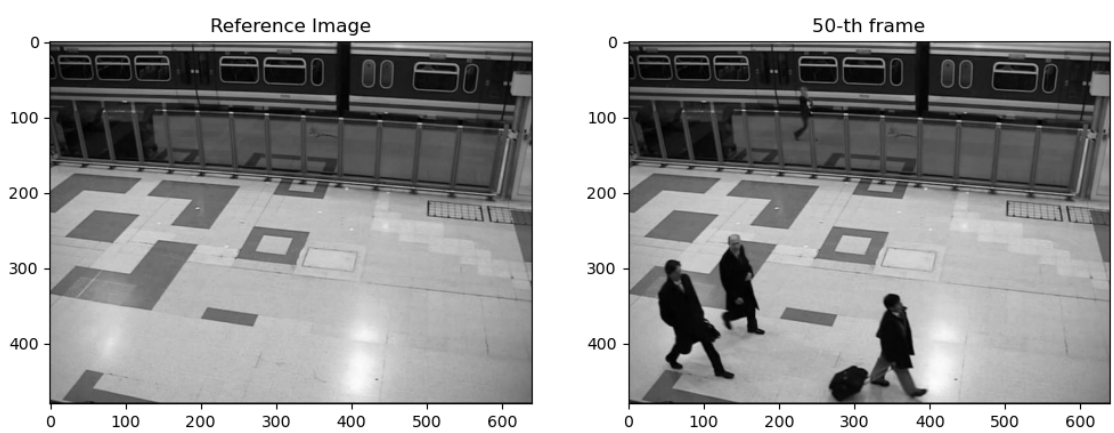
```
<IPython.core.display.Video object>
```

Then we choose the 50-th frame as the image that we are going to use to analyze the results using different threshold strategies:

[22]:
```
FRAME_IDX = 50
_frame = es1_frames[FRAME_IDX]
```

So let's plot the reference image (the "ground truth") beside of this frame in order to look at the differences:

[23]:
```
plt.figure(figsize=(12,8))
plt.subplot(121)
plt.imshow(bg_fixed,cmap='gray')
plt.title('Reference Image')
plt.subplot(122)
plt.imshow(_frame,cmap='gray')
plt.title('50-th frame')
plt.show()
```



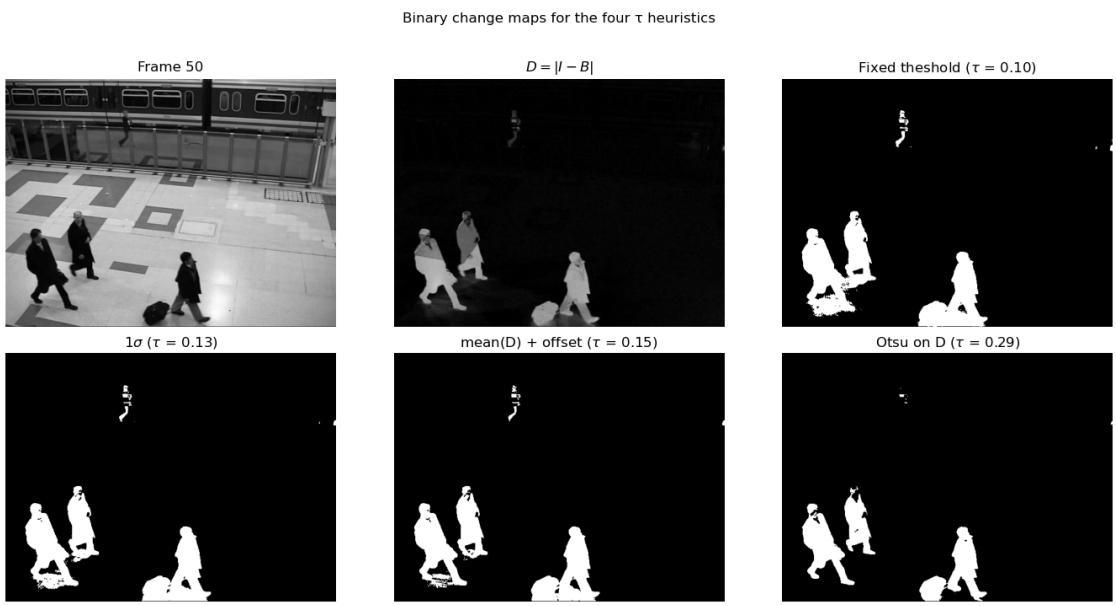
From the previous plot, we expect to be able to detect the four people that are walking in the frame. So let's run the change detection algorithm with the four different thresholds:

[24]:
```
MEAN_OFFSET = 0.1
TAU = 0.1
K_SIGMA = 1
_ref = bg_fixed

mask_tau, diff = change_detection_with_tau(_frame, _ref, TAU)
mask_s, _, tau_s = change_detection_ksigma(_frame, _ref, k=K_SIGMA)
mask_mo, _, tau_mo = change_detection_mean(_frame, _ref, MEAN_OFFSET)
mask_ot, diff_ot, tau_ot = change_detection_otsu(_frame, _ref)
```

Then we are going to plot the binary mask obtained by this algorithm with different thresholds:

[25]:
```
plot_binary_maps(_frame, FRAME_IDX, diff, mask_tau, TAU, mask_s, tau_s,␣
↪K_SIGMA, mask_mo, tau_mo, mask_ot, tau_ot)
```



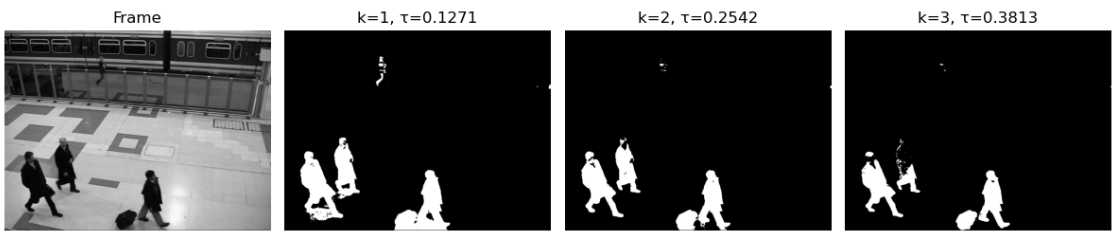
From the previous plot we can say (for each threshold chosen) that: 1. **Fixed tau:** we chose a low value of tau to preserve weak motion. It correctly detects the four people walking by, but it also keeps shadows and some background noise. In fact a low threshold classifies not only true motion but also small illumination residuals (shadow boundaries, sensor noise, slight texture flicker) as foreground. 2. **σ:** from this heuristic, we got a value of tau in between fixed tau and mean + offset (for this frame). By setting $k = 1$, the threshold is set exactly to the standard deviation (σ) of the difference image. This creates high sensitivity, ensuring better silhouette preservation of the people, but it also results in a noisy binary map that includes shadow residuals. 3. **Mean + offset:** the

resulting tau is intermediate. It keeps more foreground than the previous case while reducing part of the shadow noise. In fact, this method shifts tau according to global residual level in the frame, then uses the offset to reject low-amplitude clutter while still keeping moderate motion evidence. 4. **Otsu algorithm:** this automatically selected tau is also relatively high in this frame, which leads to under-detection of weaker changes. This is due to the fact that the histogram is not strongly bimodal here (weak motion overlaps with noise/shadow residuals).

For this specific frame, sigma tau and mean + offset give the best compromise between foreground preservation and noise suppression. In fact, it is also important to note that the results that we get from applying the change detection algorithm will be "cleaned" by the map cleaning step. So keeping more information, even though it could be meaningless, ensure that the actual blobs that we want to track are correctly detected.

Tuning k for ksigma method In this tiny section, we are going to tune the optimal value for k in the ksigma heuristic:

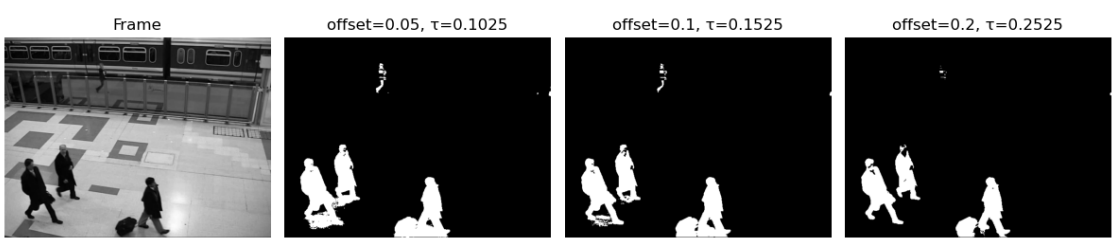
[26]:
```
plt.show(plot_tune_k_sigma(_frame, _ref, ks=(1.0, 2.0, 3.0), figsize=(12,8)))
```



With k=2 or k=3 the person next to the train is no longer detected, so for this image sequence we will be using k=1.

Tuning offset for mean + offset method In this tiny section, we are going to tune the optimal value for the offset in the mean + offset heuristic:

[27]:
```
plt.show(plot_tune_mean_offset(_frame, _ref, offsets=(0.05, 0.1, 0.2),␣
↪figsize=(12,8)))
```

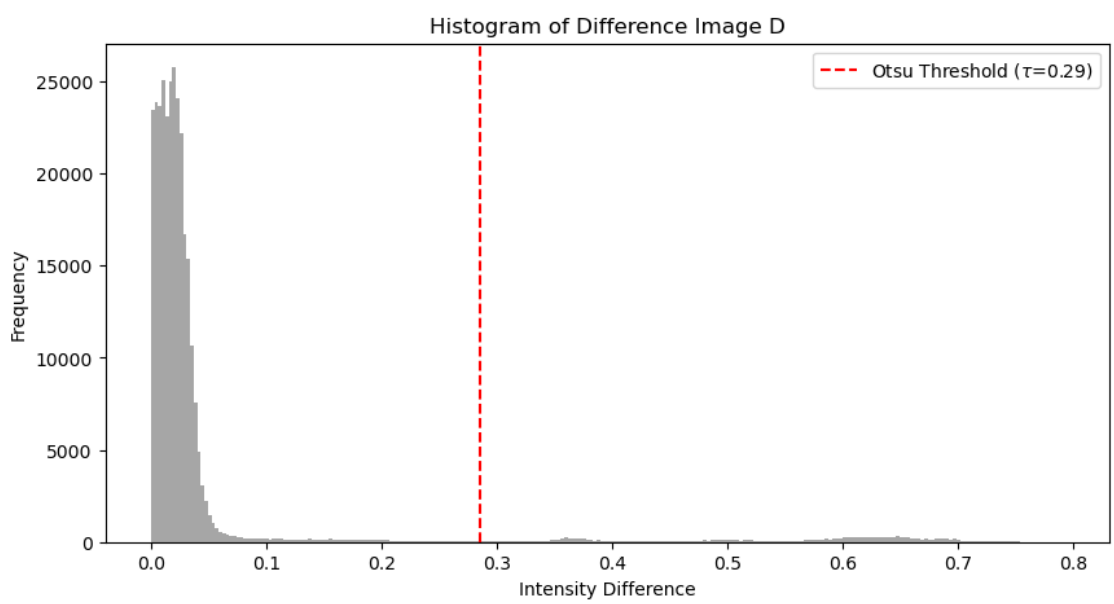


With offset=0.2, the person next to the train is no longer detected. With offset=0.05 and offset=0.1 the detection is accurate, but with the lower offset we keep most of the shadows of the people. For

this reason, we will be using offset=0.1.

Otsu analysis Otsu is fully data-driven and can work well when the histogram of $D = |I_t - I_{ref}|$ has two clearly separated modes (background noise vs moving pixels). In this section, we are going to see why the result, that we got by using the Otsu algorithm to select tau, is worse than the others:

[28]:
```
plt.figure(figsize=(10, 5))
plt.hist(diff_ot.ravel(), bins=256, color='gray', alpha=0.7)
plt.axvline(tau_ot, color='r', linestyle='--', label=fr'Otsu Threshold␣
↪($\tau$={tau_ot:.2f})')
plt.title("Histogram of Difference Image D")
plt.xlabel("Intensity Difference")
# plt.yscale('log')
plt.ylabel("Frequency")
plt.legend()
plt.show()
```



In this sequence, many changed pixels are weak and overlap with the noise mode, so Otsu tends to pick a relatively high threshold and miss part of the foreground. This is why Otsu is informative here, but not necessarily optimal for recall in this frame.

1.13.3 Map cleaning and connected components

In this section we are going to compute the third and fourth step of a blob matching algorithm, which consist of computing the cleaning of the binary map that we got in the previous step and then find the connected components of the cleaned version of the map. At the end of these steps, we will have the blobs of our frame, that could be used to track the movements over different adiacent

frames.

Let's first compute the cleaning of the binary map for all the maps that we computed before:

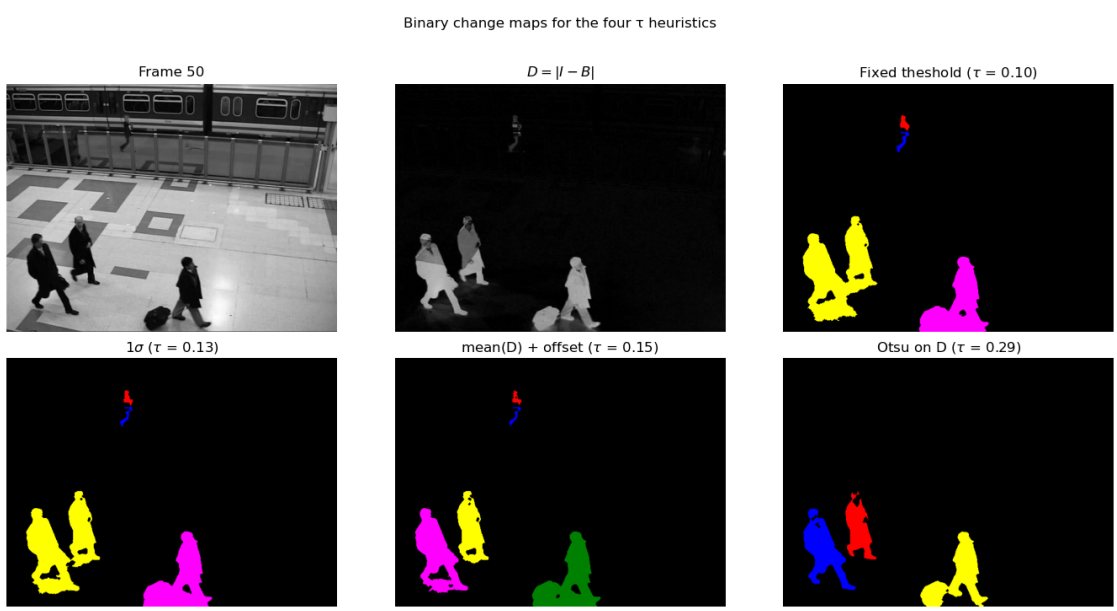
[29]:
```
changes_tau = map_cleaning(mask_tau)
changes_s = map_cleaning(mask_s)
changes_mo = map_cleaning(mask_mo)
changes_ot = map_cleaning(mask_ot)
```

Then let's compute the connected components and also use a function that colors them in order to have a clear visualization of the resulting blobs:

[30]:
```
blobs_img_tau, blob_props_tau = compute_cc(changes_tau)
colored_label_image_tau = color.label2rgb(blobs_img_tau, bg_label=0)
blobs_img_s, blob_props_s = compute_cc(changes_s)
colored_label_image_s = color.label2rgb(blobs_img_s, bg_label=0)
blobs_img_mo, blob_props_mo = compute_cc(changes_mo)
colored_label_image_mo = color.label2rgb(blobs_img_mo, bg_label=0)
blobs_img_ot, blob_props_ot = compute_cc(changes_ot)
colored_label_image_ot = color.label2rgb(blobs_img_ot, bg_label=0)
```

Lastly, we are going to plot the resulting connected components so as to write down some comments about the results:

[31]:
```
plot_binary_maps(_frame, FRAME_IDX, diff, colored_label_image_tau, TAU,␣
↪colored_label_image_s, tau_s, K_SIGMA, colored_label_image_mo, tau_mo,␣
↪colored_label_image_ot, tau_ot)
```



From the previous plot (after cleaning + connected components), we can comment similarly to

the threshold analysis: 1. **Fixed tau:** after morphology, the main person blobs are still present. Because a fixed threshold often creates "noise bridges" between nearby objects (especially if the threshold is low), the connected component function tends to group distinct people—such as the two on the left—into a single, merged blob. 2. **σ:** with $k = 1$, the high sensitivity preserves silhouettes effectively, but it is prone to capturing coherent shadow patches. 3. **Mean + offset:** gives the best compromise in this frame: people are still detected while many spurious pixels are removed. In fact, the mask is balanced enough that morphology mostly performs refinement (fill/compact) rather than aggressive correction. 4. **Otsu algorithm:** blobs are generally cleaner, but under-segmentation happened since weak-motion pixels were absorbed into background. In fact, while the weak foreground is already suppressed, the connected-component extraction sees fewer pixels per person, producing truncated blobs and occasional identity instability later.

Overall, for this example, mean + offset remains the most balanced choice before blob matching because it provides connected, sufficiently complete person silhouettes with limited clutter. One important thing to notice is that with mean + offset tau, the two persons on the left are well separated, while with the fixed tau (or sigma tau) the connected component function groups these people together.

1.13.4 Blob matching

Given the procedure explained in the related section, we will have a few of subsections that investigate: 1. The importance of the different matching strategies, in fact we will try to find the best descriptor for this image sequences by setting to 1 a particular weight descriptor while leaving all the other to 0. 2. The importance of the algorithm chosen, in fact we have multiple algorithms to run such as change_detection_* and running_average_. *In this way we will se the differences between algorithms. We will first analyze the change_detection_* algorithms and then compare these results to the ones returned by the running_average_*. 3. (Additional) The effect of using a reference background that we know it's the ground truth in comparison to the background that we could use by averaging the first frames that we assume to be empty. 4. (Additional) The effect of* alpha* in the running_average algorithms.

As a last thing, we need a way to compare the blob matching results. In fact, it's pretty difficult to spot the differences visually by looking at the output video. For this reason, the *compute_blobs_with_tracking* returns both the frames in which the tracking is highlighted but also the tracking history. These logs allow us to compare the different results that we may get running a specific algorithm and in particular they have the following features: - n_tracks: it represents the total number of unique object paths (tracks) identified by the algorithm across the entire image sequence. - mean_len: it's the average duration of a track, measured by the number of consecutive frames an object was successfully matched and tracked. - median_len: it indicates the middle value of all track lengths. - long_tracks: it counts how many tracks persisted for a significant duration, typically defined in the algorithm as being greater than or equal to 5 frames. - short_ratio: it calculates the proportion of tracks that were short-lived (fewer than 5 frames).

Different matching strategies We are going to isolate one descriptor at a time (w_* = 1, others 0) while fixing detection to manual_static ($\tau = 0.1$) on bg_fixed. Under this setup, differences in summarize_tracks just reflect assignment behavior, not background modeling.

Note: the rows are sorted by increasing short_ratio(<min_len) (fewer short-lived IDs is better), with median_len as the second most importat feature. We expect that between consecutive frames,

since the centroid displacement is physically bounded, the distance-only matching is the easiest one that we could use to get a good result. Pure *intensity*, *area*, or *eccentricity* descriptors lack that temporal smoothness and tend to swap identities.

At this point, let's compute the matching on the whole image sequence, using the algorithm *manual_static* (fixed threshold) with a threshold of 0.1 and using the "ground truth" reference image:

[32]:
```
final_frames_manual_static_dist, tracks_dist =␣
↪compute_blobs_with_tracking(es1_frames, bg_fixed, 'manual_static', tau=0.1,␣
↪w_dist=1, w_int=0, w_area=0, w_ecc=0)
final_frames_manual_static_int, tracks_int =␣
↪compute_blobs_with_tracking(es1_frames, bg_fixed, 'manual_static', tau=0.1,␣
↪w_dist=0, w_int=1, w_area=0, w_ecc=0)
final_frames_manual_static_area, tracks_area =␣
↪compute_blobs_with_tracking(es1_frames, bg_fixed, 'manual_static', tau=0.1,␣
↪w_dist=0, w_int=0, w_area=1, w_ecc=0)
final_frames_manual_static_ecc, tracks_ecc =␣
↪compute_blobs_with_tracking(es1_frames, bg_fixed, 'manual_static', tau=0.1,␣
↪w_dist=0, w_int=0, w_area=0, w_ecc=1)

tracks_by_strategy = {
    'distance': tracks_dist,
    'intensity': tracks_int,
    'area': tracks_area,
    'eccentricity': tracks_ecc,
}
```

Now we have to compute the stats for the tracking history for each all the cases calculated before:

[33]:
```
rows = []
for name, tracks in tracks_by_strategy.items():
    stats = summarize_tracks(tracks, min_len=5)
    rows.append((name, stats))

rows = sorted(rows, key=lambda x: (x[1]['short_ratio(<min_len)'],␣
↪-x[1]['mean_len']))
print('Strategy comparison (sorted by lower short_ratio, then higher mean_len):
↪\n')
print_stats(rows)
```

```
Strategy comparison (sorted by lower short_ratio, then higher mean_len):

distance        | n_tracks= 13 | mean_len=21.77 | median_len=6.00 | long_tracks=
9 | short_ratio=0.308
intensity       | n_tracks= 50 | mean_len=5.66 | median_len=2.00 | long_tracks=
22 | short_ratio=0.560
area            | n_tracks= 49 | mean_len=5.78 | median_len=3.00 | long_tracks=
18 | short_ratio=0.633
```

```
eccentricity    | n_tracks= 85 | mean_len=3.33 | median_len=2.00 | long_tracks=
24 | short_ratio=0.718
```

Distance vicinity emerged as the most robust single-feature matcher, achieving the lowest short_ratio (0.308) and the highest median_len (6.00). This confirms that centroid displacement is the most stable descriptor in this clip, as physical movement constraints reduce ambiguous assignments and help maintain long-term IDs.

The Intensity and Area strategies show significant performance degradation compared to distance, generating 50 and 49 tracks respectively, with short_ratios rising to 0.560 and 0.633. This suggests that these features are more sensitive to segmentation noise or illumination changes in this specific environment, causing pixel statistics and blob sizes to break track continuity more frequently.

Eccentricity was the least reliable strategy, producing 85 tracks and a very high short_ratio of 0.718. Because human movement involves pose changes that significantly alter the shape and orientation of the bounding box, relying on eccentricity alone leads to frequent mismatches and an excessive number of short-lived IDs.

In the next subsections, we will continue using the distance vicinity since it is clearly the superior descriptor among those tested. Below are the visual results obtained using the distance-based strategy:

[34]:
```
export_and_show_video(final_frames_manual_static_dist,␣
↪'final_frames_manual_static_dist.mp4')
```

```
Video saved as final_frames_manual_static_dist.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

Different algorithms In this subsection, every comparison below uses the same matcher (w_dist=1, other weights zero) and background. We are also going to look at the results of the manual_static algorithm in comparison with its adapative counterpart.

So let's run the comparison between *static* algorithms:

[35]:
```
ALGO_PARAMS = {'manual_static':  dict(tau=0.1), 'ksigma_static':  dict(k=1.0),␣
↪'mean_static':    dict(offset=0.1), 'otsu_static':    dict()}

algo_tracks = {}
algo_frames = {}
for method, params in ALGO_PARAMS.items():
    frames_out, tracks_out = compute_blobs_with_tracking(es1_frames, bg_fixed,␣
↪method, w_dist=1, w_int=0, w_area=0, w_ecc=0, alpha=0.1, **params,)
    algo_tracks[method] = tracks_out
    algo_frames[method] = frames_out

print('  over full sequence (same params as above):')
for method, params in ALGO_PARAMS.items():
    taus = collect_taus_sequence(es1_frames, bg_fixed, method, alpha=0.1,␣
↪**params)
```

```
    st = summarize_tau_sequence(taus)
    print(f"  {method:16s} mean  ={st['mean']:.4f}  min={st['min']:.4f} ␣
↪max={st['max']:.4f}")
```

```
 over full sequence (same params as above):
  manual_static    mean  =0.1000  min=0.1000  max=0.1000
  ksigma_static    mean  =0.1066  min=0.0700  max=0.1401
  mean_static      mean  =0.1434  min=0.1270  max=0.1556
  otsu_static      mean  =0.2776  min=0.2527  max=0.3153
```

From this table we can see that Otsu's method calculates a significantly higher mean value (0.2776) than the other heuristics, which explains its tendency toward under-segmentation by filtering out low-intensity foreground pixels. In contrast, the manual and $k\sigma$ methods utilize lower thresholds (≈ 0.10) to preserve the subtle silhouettes necessary for stable tracking. While the manual threshold remains fixed, the dynamic heuristics ($k\sigma$ and mean-offset) successfully adapt to per-frame noise variations.

At this point we are going to show the stats for each tracking history so as to do some comments about the results:

[36]:
```
algo_rows = []
for method, tracks in algo_tracks.items():
    stats = summarize_tracks(tracks, min_len=5)
    algo_rows.append((method, stats))

algo_rows = sorted(algo_rows, key=lambda x: (x[1]['short_ratio(<min_len)'],␣
↪-x[1]['median_len']))

print('Algorithm comparison with fixed background:\n')
print_stats(algo_rows)
```

```
Algorithm comparison with fixed background:

manual_static   | n_tracks= 13 | mean_len=21.77 | median_len=6.00 | long_tracks=
9 | short_ratio=0.308
mean_static     | n_tracks= 23 | mean_len=12.87 | median_len=6.00 | long_tracks=
14 | short_ratio=0.391
otsu_static     | n_tracks= 11 | mean_len=18.45 | median_len=7.00 | long_tracks=
6 | short_ratio=0.455
ksigma_static   | n_tracks= 20 | mean_len=14.85 | median_len=2.00 | long_tracks=
7 | short_ratio=0.650
```

Given the tracking history stats, we can see that the manual_static algorithm is the most effective strategy for this specific sequence regarding identity stability, as it achieved the lowest short_ratio (0.308). This indicates that it produced the fewest short-lived or noisy tracks while maintaining the highest mean_len (21.77). By manually selecting a threshold, it provided a superior balance between preserving foreground silhouettes and suppressing background clutter compared to the automated heuristics in this instance. The mean_static approach falls in the middle of the group with a short_ratio of 0.429. Its permissive nature may retain enough background noise to inflate

the number of short-lived tracks compared to a tuned manual threshold.

In general, we observe the fundamental trade-off in change detection: - Strict thresholds result in "cleaner" masks but drop weak foreground data, leading to fragmented trajectories and shorter track lengths. - Permissive thresholds (like Mean + Offset) might result in a higher total track count but they ensure the matching algorithm persistent regions over time, allowing individual IDs to survive much longer.

At this point, let's compare the results of the *static* algorithms (change_detection_*) with respect to the* adaptive* ones (running_average_*). In order to do so, we are going to look at the differences (deltas) in the *short_ratio* (short) and *median_len* (med) stats:

[37]:
```
# Compare each algorithm with fixed vs adaptive background
BG_COMPARISON = [
    ('manual', dict(static='manual_static', adaptive='manual_adaptive', tau=0.
 ↪1)),
    ('ksigma', dict(static='ksigma_static', adaptive='ksigma_adaptive', k=1)),
    ('mean',   dict(static='mean_static', adaptive='mean_adaptive', offset=0.
 ↪1)),
    ('otsu',   dict(static='otsu_static', adaptive='otsu_adaptive')),
]

bg_rows = []
for name, cfg in BG_COMPARISON:
    params = {k: v for k, v in cfg.items() if k not in ('static', 'adaptive')}

    _, tracks_static = compute_blobs_with_tracking(es1_frames, bg_fixed,␣
 ↪cfg['static'], w_dist=1, w_int=0, w_area=0, w_ecc=0, alpha=0.1, **params)
    _, tracks_adapt = compute_blobs_with_tracking(es1_frames, bg_fixed,␣
 ↪cfg['adaptive'], w_dist=1, w_int=0, w_area=0, w_ecc=0, alpha=0.1, **params)

    s_static = summarize_tracks(tracks_static, min_len=5)
    s_adapt = summarize_tracks(tracks_adapt, min_len=5)

    bg_rows.append((name, s_static, s_adapt))

print('Fixed vs adaptive background comparison:\n')
for name, s0, s1 in bg_rows:
    d_short = s1['short_ratio(<min_len)'] - s0['short_ratio(<min_len)']
    d_med = s1['median_len'] - s0['median_len']
    print(
        f"{name:8s} | "
        f"static(short={s0['short_ratio(<min_len)']:.3f}, med={s0['median_len']:
 ↪.2f}) -> "
        f"adaptive(short={s1['short_ratio(<min_len)']:.3f},␣
 ↪med={s1['median_len']:.2f}) | "
        f"Δshort={d_short:+.3f}, Δmed={d_med:+.2f}"
    )
```

```
Fixed vs adaptive background comparison:

manual   | static(short=0.308, med=6.00) -> adaptive(short=0.267, med=15.00) |
Δshort=-0.041, Δmed=+9.00
ksigma   | static(short=0.650, med=2.00) -> adaptive(short=0.312, med=9.50) |
Δshort=-0.338, Δmed=+7.50
mean     | static(short=0.391, med=6.00) -> adaptive(short=0.304, med=8.00) |
Δshort=-0.087, Δmed=+2.00
otsu     | static(short=0.455, med=7.00) -> adaptive(short=0.529, med=4.00) |
Δshort=+0.075, Δmed=-3.00
```

From the previous table, we can see that the ksigma algorithm benefited the most from switching to an adaptive background. Its short_ratio decreased significantly from 0.650 to 0.312 ($\Delta short = -0.338$), and the median track length improved by 7.50, indicating that tracks became much more robust. This is likely because the static ksigma heuristic often acts as a strict gate that drops weak foreground pixels. By allowing the background to adapt, the model better handles gradual changes and prevents those weak pixels from causing track breaks. Also note the impact of the adaptive background on the performances of the manual_static algorithm. In contrast, the otsu method showed a regression in performance when using an adaptive background.

As a last thing for this subsection, we are going to select manual_static algorithm (with tau=0.1) and compare it visually with the adaptive counterpart:

[38]:
```
frames_static, _ = compute_blobs_with_tracking(
    es1_frames, bg_fixed, 'manual_static',
    w_dist=1, w_int=0, w_area=0, w_ecc=0,
    alpha=0.1, tau=0.1
)
frames_adapt, _ = compute_blobs_with_tracking(
    es1_frames, bg_fixed, 'manual_adaptive',
    w_dist=1, w_int=0, w_area=0, w_ecc=0,
    alpha=0.1, tau=0.1
)

side_by_side_frames = side_by_side_video(frames_static, frames_adapt, f'STATIC:␣
↪manual_static', f'ADAPTIVE: manual_adaptive')
export_and_show_video(side_by_side_frames, 'best_static_vs_adaptive.mp4')
```

```
Video saved as best_static_vs_adaptive.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

As we said before, it's pretty difficult to comment the results with a video. In order to have a grasp of the differences we can stop the video at certain frames and look at the blobs and related IDs returned by the two algorithms.

1.13.5 (Additional) Ground truth background and averaged one

When we were importing the frames, we saw that the provided folder had one *ground truth* background and two empty frames that have been averaged to get a reference image. In this subsection, we are going to look at the differences between the results of the blob matching algorithm, since in the averaged image there were a backpack that is not present in the ground truth reference.

First we are going to run the static algorithms to both reference images, so as to look at the tracking stats. We are expecting that since there's a backpack which is not present in any of the frames of the video, that item will be always detected as a blob resulting in "better" stats that realistically is a problem. Then we are going to use the adaptive version of the best algorithm in order to remove this expected behaviour.

Let's start by running the static algorithms:

[39]:
```
BG_INIT_METHODS = {
    'manual_static': {'tau': 0.1},
    'ksigma_static': {'k': 1.0},
    'mean_static': {'offset': 0.1},
    'otsu_static': {},
}

bg_init_rows = []
bg_init_outputs = {}

for method, params in BG_INIT_METHODS.items():
    frames_fix, tracks_fix = compute_blobs_with_tracking(
        es1_frames, bg_fixed, method,
        w_dist=1, w_int=0, w_area=0, w_ecc=0,
        alpha=0.1, **params
    )
    frames_avg, tracks_avg = compute_blobs_with_tracking(
        es1_frames, bg_avg, method,
        w_dist=1, w_int=0, w_area=0, w_ecc=0,
        alpha=0.1, **params
    )

    s_fix = summarize_tracks(tracks_fix, min_len=5)
    s_avg = summarize_tracks(tracks_avg, min_len=5)

    bg_init_rows.append((method, s_fix, s_avg))
    bg_init_outputs[method] = {
        'frames_fixed': frames_fix,
        'frames_avg': frames_avg,
        'stats_fixed': s_fix,
        'stats_avg': s_avg,
    }

print('bg_fixed vs bg_avg (same algorithm, same matching):\n')
```

```
for method, s_fix, s_avg in bg_init_rows:
    d_short = s_avg['short_ratio(<min_len)'] - s_fix['short_ratio(<min_len)']
    d_med = s_avg['median_len'] - s_fix['median_len']
    print(
        f"{method:13s} | "
        f"fixed(short={s_fix['short_ratio(<min_len)']:.3f},␣
 ↪med={s_fix['median_len']:.2f}) -> "
        f"avg(short={s_avg['short_ratio(<min_len)']:.3f},␣
 ↪med={s_avg['median_len']:.2f}) | "
        f"Δshort(avg-fixed)={d_short:+.3f}, Δmed={d_med:+.2f}"
    )
```

```
bg_fixed vs bg_avg (same algorithm, same matching):

manual_static | fixed(short=0.308, med=6.00) -> avg(short=0.231, med=18.00) |
Δshort(avg-fixed)=-0.077, Δmed=+12.00
ksigma_static | fixed(short=0.650, med=2.00) -> avg(short=0.312, med=10.50) |
Δshort(avg-fixed)=-0.338, Δmed=+8.50
mean_static   | fixed(short=0.391, med=6.00) -> avg(short=0.333, med=9.00) |
Δshort(avg-fixed)=-0.058, Δmed=+3.00
otsu_static   | fixed(short=0.455, med=7.00) -> avg(short=0.417, med=6.50) |
Δshort(avg-fixed)=-0.038, Δmed=-0.50
```

The comparison between bg_fixed and bg_avg reveals that using an averaged background consistently improves tracking stability by accurately capturing the specific lighting and sensor noise of the recording. As previously noted, the "ghost" backpack in bg_avg likely artificially boosts these metrics by creating a persistent, non-fragmenting track.

The ksigma_static heuristic showed the most dramatic improvement, with its short_ratio dropping by 0.338 and median_len increasing by 8.50. This demonstrates that statistical thresholds are far more effective when the background model's noise distribution closely matches the actual video data.

While manual_static achieved the highest longevity boost (+12.00), mean_static maintained its role as a robust data-driven compromise with a very low fragmentation ratio. Using a sensor-accurate initialization like bg_avg ensures that even simple heuristics can better preserve silhouette coherence over time.

So let's visualize the results:

[40]:
```
# Side-by-side visualization of same method with bg_fixed vs bg_avg
vis_data = bg_init_outputs['manual_static']
frames_fix = vis_data['frames_fixed']
frames_avg = vis_data['frames_avg']

bg_init_side_by_side = side_by_side_video(frames_fix, frames_avg, f'bg_fixed |␣
 ↪manual_static', f'bg_avg | manual_static')
export_and_show_video(bg_init_side_by_side, 'bg_fixed_vs_bg_avg_best_method.
 ↪mp4')
```

```
Video saved as bg_fixed_vs_bg_avg_best_method.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

As expected, in the right hand side video, there is the "ghost" backpack in all the frames of the sequence.

1.13.6 (Additional) Effect of alpha on running-average algorithm

In this subsection, we are going to analyze how alpha changes stability. Using a small value of alpha updates background slowly (more memory), while with a large alpha the background adapts quickly to the current frame (less memory). In order to do so, we are going to use just the mean + offset method, taking into account that the correct value of alpha changes with respect to the algorithm chosen.

[41]:
```
ALPHAS = [0.01, 0.03, 0.05, 0.1, 0.2]
alpha_sweep_results = []

print('Alpha sweep for running-average (mean + offset) algorithm:\n')

for a in ALPHAS:
    _, tracks = compute_blobs_with_tracking(
        es1_frames,
        bg_fixed,
        method='mean_adaptive',
        offset=0.1,
        w_dist=1, w_int=0, w_area=0, w_ecc=0,
        alpha=a
    )

    stats = summarize_tracks(tracks, min_len=5)
    alpha_sweep_results.append((f"alpha={a}", stats))

print_stats(alpha_sweep_results)
```

```
Alpha sweep for running-average (mean + offset) algorithm:

alpha=0.01      | n_tracks= 21 | mean_len=14.38 | median_len=9.00 | long_tracks=
14 | short_ratio=0.333
alpha=0.03      | n_tracks= 19 | mean_len=16.26 | median_len=10.00 |
long_tracks= 14 | short_ratio=0.263
alpha=0.05      | n_tracks= 18 | mean_len=17.22 | median_len=10.50 |
long_tracks= 14 | short_ratio=0.222
alpha=0.1       | n_tracks= 23 | mean_len=14.39 | median_len=8.00 | long_tracks=
16 | short_ratio=0.304
alpha=0.2       | n_tracks= 27 | mean_len=13.93 | median_len=8.00 | long_tracks=
17 | short_ratio=0.370
```

For this specific image sequence, alpha = 0.05 provides the best balance. It achieves the lowest

short_ratio (0.222) and the highest median_len (10.50), indicating that silhouettes are preserved long enough for the matching algorithm to maintain stable, persistent identities.

As alpha increases to 0.1 and 0.2, tracking quality decreases. The short_ratio rises significantly to 0.370 at alpha = 0.2. This is because a high alpha causes the background model to update too quickly, "absorbing" for example the shadows of the people walking by. This results in the loss of the foreground silhouette and the fragmentation of a single person's path into several short, disconnected tracks.

At a very low value like alpha = 0.01, the system also experiences higher fragmentation (short_ratio = 0.333). This suggests that extremely slow adaptation is insufficient to incorporate subtle lighting changes, leading to more "salt-and-pepper" noise being detected as foreground, which creates short-lived, noisy tracks.

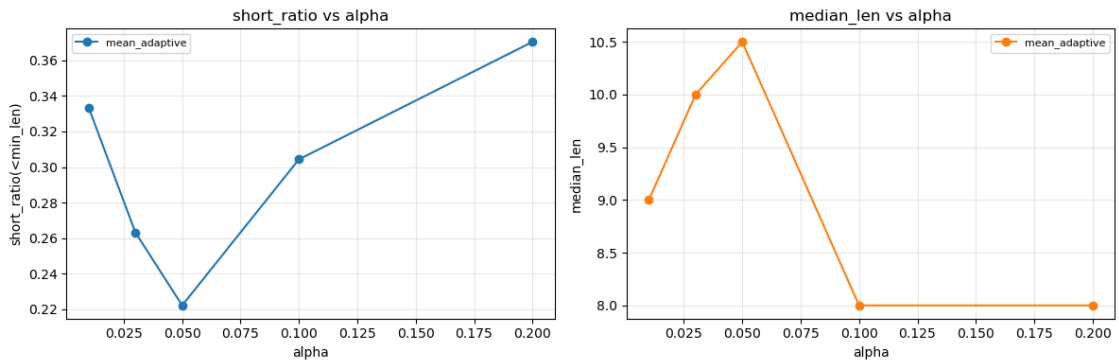
In order to have a better grasp of these results, let's plot them:

[42]:
```
# Visual summary
plt.figure(figsize=(12,4))
xs = ALPHAS
stats_list = [item[1] for item in alpha_sweep_results]

# Subplot 1: short_ratio vs alpha
plt.subplot(1,2,1)
ys_short = [s['short_ratio(<min_len)'] for s in stats_list]
plt.plot(xs, ys_short, marker='o', label='mean_adaptive', color='tab:blue')
plt.title('short_ratio vs alpha')
plt.xlabel('alpha')
plt.ylabel('short_ratio(<min_len)')
plt.grid(True, alpha=0.3)
plt.legend(fontsize=8)

# Subplot 2: median_len vs alpha
plt.subplot(1,2,2)
ys_median = [s['median_len'] for s in stats_list]
plt.plot(xs, ys_median, marker='o', label='mean_adaptive', color='tab:orange')
plt.title('median_len vs alpha')
plt.xlabel('alpha')
plt.ylabel('median_len')
plt.grid(True, alpha=0.3)
plt.legend(fontsize=8)

plt.tight_layout()
plt.show()
```



Here we can clearly see that with alpha = 0.05, we get the best results between short_ratio and median_len.

1.14 Second video

We are asked to acquire a video, having in mind the type of experiments we want to perform and the challenges we are interested in addressing. The video that I recorded shows a corner of my room as the background, with some random object in it so as to not make the background too plain. Then this video contains the following sections: 1. I threw two big sports balls in the frame: I want to see how Otsu performes in this subsection of the frame, since these objects are pretty big with respect to the frame. I expect that in this case the histogram will have a bi-modular shape and the threshold works fine. 2. I threw a small toy car and a big one: I want to analyze the blob matching procedure with objects that does not have the same shape/eccentricity. Also the small toy car is quite fast, so I want to see how the matching results of it. 3. I threw two big toy cars that collide: I want to see how the matching handles this situation. 4. I threw a small toy car that stops at in the frame and remains there till to the end of the video: I want to see how the algorithms handle an object which was not present at the beginning but it will be present from a certain frame onwards. Also I want to perform the tracking on this sequence.

Given this image sequence, the meta-question that I'm interested in is to analyze performances of the different matching strategies, since in this case the objects do not have the same shape or area (which was a case in the first video).

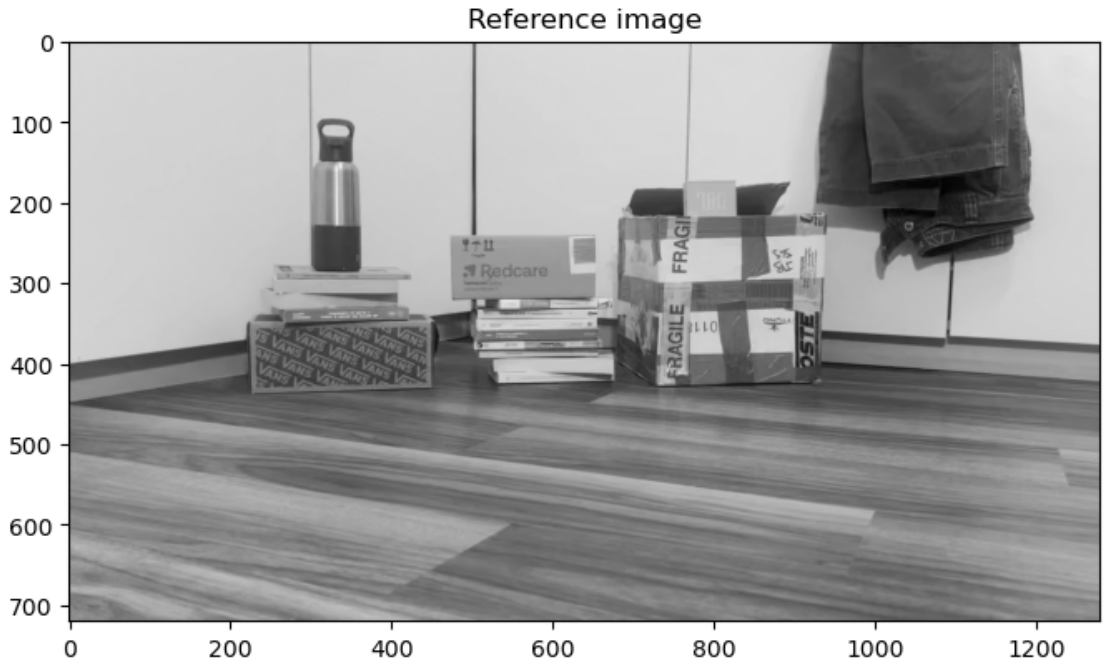
Next, we run the same threshold comparison as in the first video on es2_frames, then split the analysis into four segments aligned with the investigation that I'm interested in.

1.14.1 Background initialization

So let's get started and let's import the background:

[43]:
```
bg_fixed_2nd = preprocessing(io.imread(PATH2 + 'Background.jpg'))
```

[44]:
```
plt.figure(figsize=(8,6))
plt.imshow(bg_fixed_2nd,cmap='gray')
plt.title('Reference image')
plt.show()
```



Note that we are going to use a fixed frame gathered when I have recorded the video.

1.14.2 Import frames

At this point, we need to import the frames of the video:

[45]:
```
es2_frames = import_images(PATH2 + "video/")
```

In order to be more clear, let's also display the video:

[46]:
```
export_and_show_video(es2_frames, 'original_es2.mp4')
```

```
Video saved as original_es2.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

1.14.3 Change detection and different thresholds (second video)

Before proceding with the full pipeline explained before, let's show the results of the change_detection algorithms on a frame of this video:

[47]:
```
_ref2 = bg_fixed_2nd
MEAN_OFFSET_ES2 = 0.1
TAU_ES2 = 0.1
K_SIGMA_ES2 = 2
```

```
n2 = len(es2_frames)
FRAME_IDX_ES2 = 220
_frame2 = es2_frames[FRAME_IDX_ES2]

mask_tau2, diff2 = change_detection_with_tau(_frame2, _ref2, TAU_ES2)
mask_s2, _, tau_s2 = change_detection_ksigma(_frame2, _ref2, k=K_SIGMA_ES2)
mask_mo2, _, tau_mo2 = change_detection_mean(_frame2, _ref2, MEAN_OFFSET_ES2)
mask_ot2, _, tau_ot2 = change_detection_otsu(_frame2, _ref2)

plot_binary_maps(
    _frame2, FRAME_IDX_ES2, diff2,
    mask_tau2, TAU_ES2, mask_s2, tau_s2, K_SIGMA_ES2,
    mask_mo2, tau_mo2, mask_ot2, tau_ot2,
)
```

We can see that the overall values of the thresholds are quite similar one with respect to each other. This could be due to the fact that the video acquired has less intensity value changes between the current frame and the reference background compared to the first video. In general, all the blobs of this frame are correcly detected and in this particular case we could use any of the heuristics/algorithms defined before.

1.14.4 Investigating challenges

In order to investigate the challenges that we are interested in, we have to split the image sequence into four different parts. In fact we are interested in the following sections: 1. Frames in which there are the two sports balls. 2. Frames from the moment in which the tiny toy car join the scene. 3. Frames in which the two different sized toy cars appear on the scene. 4. Frames where the two toy cars collide.

Note that we will hard-code the range of frames that we are interested in.

[48]:
```
# Hard-coded split
sport_split = es2_frames[:69]
tiny_split = es2_frames[98:]
cars_split = es2_frames[192:245]
collide_split = es2_frames[307:390]
```

Otsu performances In this subsection, we want to investigate if the Otsu algorithm could be used in order to detect the changes of the current frame with respect to the reference image. As we saw in the previous section, Otsu algorithm worked pretty fine for a particular frame and now we want to investigate if it could be applied to the subset of the frames in which there are the two big sport balls.

Let's first show the image sequence that we will be using for our analysis:

[49]:
```
export_and_show_video(sport_split, 'sport.mp4')
```

```
Video saved as sport.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

In the first video, the Otsu algorithm returned a pretty high value for the threshold and that led to under-detection of the tiny changes of the current frame. This was due to the fact that the histogram of the difference between the current frame and the reference image was not following a bi-modular shape. For this reason, I decided to throw two big objects in the image sequence so as to analyze the "shape" of the histogram.

Before showing the histograms over this sequence, let's first show the binary maps that we get from applying the change_detection_otsu on this video:

[50]:
```
current_bg = bg_fixed_2nd.copy()
output_frame = []
for frame in sport_split:
    mask, _, _ = change_detection_otsu(frame, current_bg)
    cleaned = map_cleaning(mask)
    output_frame.append(1 * cleaned.astype(float))
export_and_show_video(output_frame, 'sport_otsu.mp4')
```

```
Video saved as sport_otsu.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

The two balls are detected throughout the sequence, but they are not perfectly captured due to the fact that the threshold chosen by the Otsu algorithm is too high. Let's visualize the difference using a lower value for the threshold:

[51]:
```
current_bg = bg_fixed_2nd.copy()
output_frame_mean = []
for frame in sport_split:
```

```
    mask, _ = change_detection_with_tau(frame, current_bg, tau=0.1)
    cleaned = map_cleaning(mask)
    output_frame_mean.append(1 * cleaned.astype(float))
otsu_mean_side_by_side = side_by_side_video(output_frame, output_frame_mean,␣
 ↪f'Otsu', f'Fixed tau')
export_and_show_video(otsu_mean_side_by_side, 'otsu_mean_side_by_side.mp4')
```

```
Video saved as otsu_mean_side_by_side.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

We can see that with a lower fixed tau the results slightly improved but there is still some areas of the balls that are not detected and this is due to the fact that in a grey-scale regime, the balls and the background share almost the same intensity values.

But let's have a loot at the Otsu threshold and histograms in 6 different moments of the sequence:

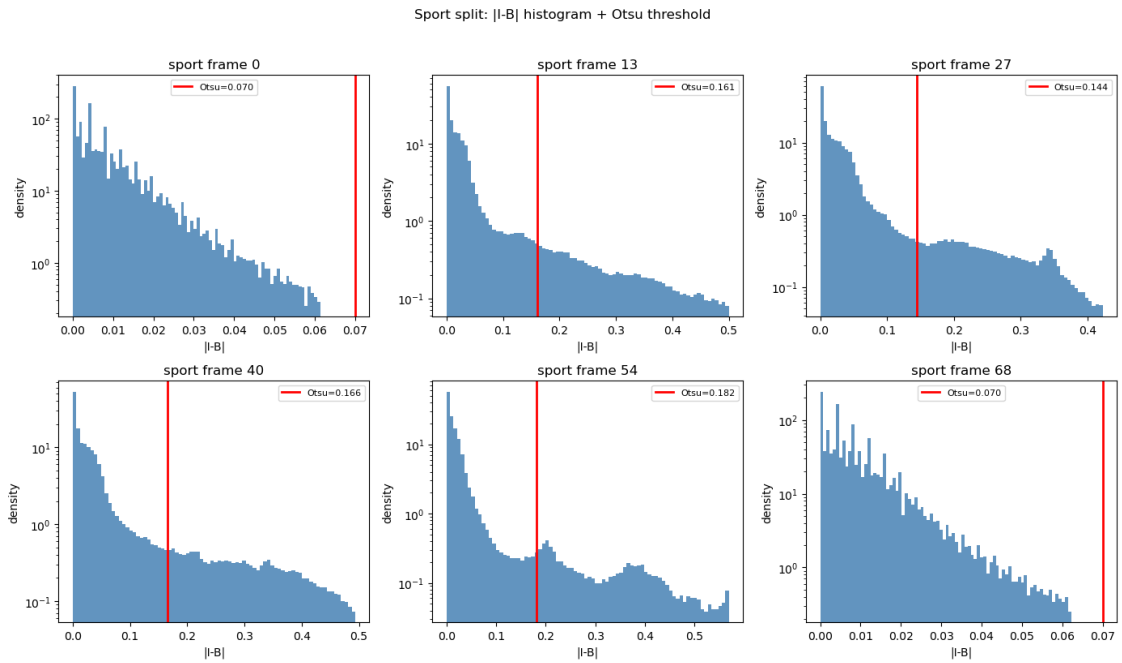
[52]:
```
# Difference histograms on sport_split to motivate Otsu
ref_bg = bg_fixed_2nd.copy()
sample_idx = np.linspace(0, len(sport_split)-1, num=min(6, len(sport_split)),␣
 ↪dtype=int)

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
axes = axes.ravel()

for ax, k in zip(axes, sample_idx):
    d = difference_map(sport_split[k], ref_bg)
    vals = d.ravel()
    hi = max(float(np.percentile(vals, 99.5)), 1e-6)
    ax.hist(vals, bins=80, range=(0, hi), density=True, color='steelblue',␣
 ↪alpha=0.85)
    t_otsu = tau_otsu(d)
    ax.axvline(t_otsu, color='red', linewidth=2, label=f'Otsu={t_otsu:.3f}')
    ax.set_title(f'sport frame {k}')
    ax.set_yscale('log')
    ax.set_xlabel('|I-B|')
    ax.set_ylabel('density')
    ax.legend(fontsize=8)

for ax in axes[len(sample_idx):]:
    ax.axis('off')

plt.suptitle('Sport split: |I-B| histogram + Otsu threshold', y=1.02)
plt.tight_layout()
plt.show()
```



We used a log-scale for these plots, but in frames 13, 27, 40, and 54, the histograms are not clearly bimodal. Even where tiny peaks exist (frame 27, 40 and 54), the foreground and background overlap too much for the Otsu algorithm to find a good "valley". This leads to a threshold that is too high, causing under-segmentation and truncated silhouettes.

Based on the results from both videos, we can conclude that Otsu applicability for change detection is limited by the specific characteristics of the scene's histogram. Otsu's algorithm assumes a bimodal histogram with two distinct peaks (background vs. foreground). In these videos, the "weak motion" of the objects and the presence of noise/shadows caused these two distributions to overlap. This under-segmentation resulted in truncated silhouettes and fragmented blobs, which often caused identity instability during the tracking and matching phase.

Tracking of different sized objects In this subsection, we investigate the tracking of objects with varying sizes. In the first video (the train station sequence), size differences were primarily a result of perspective, such as when an individual appeared smaller while walking further from the camera.

Instead in this new sequence, we evaluate whether the tracking remains accurate under these conditions and if the methods can effectively handle objects moving at high speeds. Note that in this sequence there is going to be an additional tiny car which was added at some part of the video (is not present in the reference image).

Let's have a look at the image sequence that we are going to work on:

[53]:
```
export_and_show_video(cars_split, 'cars_split.mp4')
```

```
Video saved as cars_split.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

So let's have a look at the resulting tracking obtained by using the manual_static algorithm (with tau=0.1) and using just the distance descriptor (centroid vicinity):

[54]:
```
cars_frames_track, cars_tracks = compute_blobs_with_tracking(
    cars_split, bg_fixed_2nd, method='manual_static', min_area=200, tau=0.1,
    w_dist=1, w_int=0, w_area=0, w_ecc=0
)
export_and_show_video(cars_frames_track, 'cars_split_tracking_from_tiny_bg.mp4')
```

```
Video saved as cars_split_tracking_from_tiny_bg.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

In this sequence, the high speed of the toy cars leads to fragmented tracking when relying solely on centroid vicinity for matching. Because the objects move significantly between frames, the spatial distance alone is often insufficient to maintain a consistent link. To address this, we incorporate a more complex matching cost that includes area and eccentricity:

[55]:
```
cars_frames_track_mixed, _ = compute_blobs_with_tracking(
    cars_split, bg_fixed_2nd, method='manual_static', min_area=200, tau=0.1,
    w_dist=1, w_int=0, w_area=0.5, w_ecc=0.5
)
cars_side_by_side = side_by_side_video(cars_frames_track,␣
 ↪cars_frames_track_mixed, f'JUST centroid', f'Centroid + Area + Eccentricity')
export_and_show_video(cars_side_by_side, 'cars_side_by_side.mp4')
```

```
Video saved as cars_side_by_side.mp4
Conversion successful! Displaying video…
```

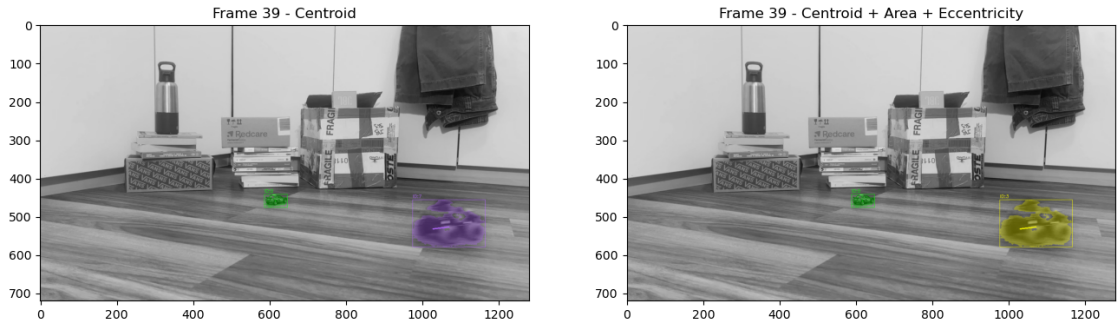
```
<IPython.core.display.Video object>
```

At first glance, the results may seem similar, however, examining the final frames reveals that the tracking based only on centroid distance produces a much higher ID value for the large toy car. In contrast, the video on the right, which uses multiple features, keeps the ID count low. This shows that adding shape and size descriptors to the matching cost significantly improves tracking stability. By ensuring that a blob in the current frame is matched to one with a similar area and eccentricity in the previous frame, the algorithm effectively prevents identity fragmentation and maintains a continuous trajectory even for fast-moving objects.

Here's an example:

[56]:
```
plt.figure(figsize=(16,14))
plt.subplot(121)
plt.imshow(cars_frames_track[39],cmap='gray')
plt.title('Frame 39 - Centroid')
plt.subplot(122)
plt.imshow(cars_frames_track_mixed[39],cmap='gray')
plt.title('Frame 39 - Centroid + Area + Eccentricity')
```

```
plt.show()
```



Collision of objects In this section, we are going to analyze the tracking performance when two blobs collide in the image plane. This test is interesting to understand how the system manages object identities when multiple silhouettes merge into a single connected component, a common challenge that often leads to identity swaps or fragmentation. Let's first see the image sequence that we will work on:

[57]:
```
export_and_show_video(collide_split, 'collide_split.mp4')
```

```
Video saved as collide_split.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

At this point let's compute the tracking using the ksigma_static (with k=1) using the same descriptors as the previous section:

[58]:
```
collision_frames_track, collision_tracks = compute_blobs_with_tracking(
    collide_split, bg_fixed_2nd, method='ksigma_static', min_area=250, k=1,
    w_dist=1, w_int=0, w_area=0.5, w_ecc=0.5
)
export_and_show_video(collision_frames_track,␣
 ↪'collision_split_tracking_from_tiny_bg.mp4')
```

```
Video saved as collision_split_tracking_from_tiny_bg.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

We can observe that the tracking is perfectly stable before the collision occurs. However, when the third car makes contact with the second car, their silhouettes merge, and the system detects them as a single, larger object. Because this combined blob has a significantly different area and centroid position compared to the individual objects in the previous frame, the matching algorithm assigned a new ID. After the collision ends and the cars separate, the system correctly identifies them as individual blobs again, and the tracking remains constant for the remainder of the sequence.

Full video and changing background In this subsection, we investigate the background update process using the running_average algorithm and evaluate the tracking performance across the entire video to determine which descriptor provides the most stable results. Let's first show the video:

[59]:
```
export_and_show_video(tiny_split, 'tiny_split.mp4')
```

```
Video saved as tiny_split.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

First of all, we are going to analyze how the background updates with the running_average algorithm:

[60]:
```
ALPHA_TINY = 0.05
OFFSET_TINY = 0.08
DELTA = 5

def run_tiny_background_update(frames, bg_init, alpha=0.05, offset=0.08,␣
↪version=1):
    bg = bg_init.copy()
    bg_snapshots = []
    fg_masks = []
    prev_fr = frames[0]
    for idx, fr in enumerate(frames):
        if idx > DELTA:
            prev_fr = frames[idx - DELTA]
        else:
            prev_fr = frames[idx]
        if version == 1:
            m, bg = running_average_with_tau(fr, bg, tau=offset, alpha=alpha)
        else:
            m, bg = running_average_with_tau_temporal(fr, bg, prev_fr,␣
↪tau=offset, tau_prime=None, alpha=0.1)
        fg_masks.append(m)
        # collect sparse snapshots for visualization
        bg_snapshots.append(bg.copy())
        prev_fr = fr
    return bg, fg_masks, bg_snapshots

bg_after_tiny, tiny_masks, tiny_bg_snaps =␣
↪run_tiny_background_update(tiny_split, bg_fixed_2nd, alpha=ALPHA_TINY,␣
↪offset=OFFSET_TINY, version=1)

export_and_show_video(tiny_bg_snaps, 'tiny_bg_snaps.mp4')
```

```
Video saved as tiny_bg_snaps.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

Although a toy car remains stationary in the middle of the scene, it does not become part of the background. This is because the running_average function is designed to incorporate smooth, slight variations in light intensity.

The algorithm follows a conditional logic: if the difference between the current frame and the reference is greater than the threshold ($|I_t - B_{t-1}| > \tau$), the background is not updated. Since the toy car represents a significant intensity change that is not coherent with the background, the change detection mask remains "true" for those pixels, preventing the background from absorbing the object over time, resulting in a persistent blob in the frame. For this reason, we have defined a function that implements not slow variations in the background and now we are going to see how it performs:

[61]:
```
bg_after_tiny, tiny_masks, tiny_bg_snaps =␣
↪run_tiny_background_update(tiny_split, bg_fixed_2nd, alpha=ALPHA_TINY,␣
↪offset=OFFSET_TINY, version=2)

export_and_show_video(tiny_bg_snaps, 'tiny_bg_snaps_adapt.mp4')
```

```
Video saved as tiny_bg_snaps_adapt.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

We can clearly see that the tiny toy car gets integrated to the background.

As a last experiment, let's compare the results of the blob tracking using different descriptors (as we did with the train station video):

[62]:
```
# Full video: compare matching strategies as before
ES2_STRATEGIES = {
    'distance':     dict(w_dist=1, w_int=0, w_area=0, w_ecc=0),
    'intensity':    dict(w_dist=0, w_int=1, w_area=0, w_ecc=0),
    'area':         dict(w_dist=0, w_int=0, w_area=1, w_ecc=0),
    'eccentricity': dict(w_dist=0, w_int=0, w_area=0, w_ecc=1),
}

es2_results = {}
rows = []
for name, wcfg in ES2_STRATEGIES.items():
    f_out, t_out = compute_blobs_with_tracking(
        tiny_split, bg_fixed_2nd, method='manual_static', min_area=250, tau=0.
↪1, **wcfg
    )
    s = summarize_tracks(t_out, min_len=5)
    es2_results[name] = {'frames': f_out, 'tracks': t_out, 'stats': s}
    rows.append((name, s))
```

```
rows = sorted(rows, key=lambda x: (x[1]['short_ratio(<min_len)'],␣
↪-x[1]['median_len']))
print('Second video strategy comparison:')
print_stats(rows)
```

```
Second video strategy comparison:
distance        | n_tracks= 26 | mean_len=15.65 | median_len=2.00 | long_tracks=
10 | short_ratio=0.615
intensity       | n_tracks= 45 | mean_len=9.04 | median_len=2.00 | long_tracks=
10 | short_ratio=0.778
area            | n_tracks= 40 | mean_len=10.18 | median_len=1.00 | long_tracks=
8 | short_ratio=0.800
eccentricity    | n_tracks= 61 | mean_len=6.67 | median_len=1.00 | long_tracks=
7 | short_ratio=0.885
```

For this specific video, spatial distance remains the most robust primary descriptor. While auxiliary costs like area and eccentricity were shown to be helpful in resolving specific issues like high-speed movement or collisions, relying on them as primary matching criteria increases identity fragmentation.

As a last thing, let's show the results:

[63]:
```
best_name = rows[0][0]
print('Best strategy on second video:', best_name)
export_and_show_video(es2_results[best_name]['frames'],␣
↪f'es2_best_tracking_{best_name}.mp4')
```

```
Best strategy on second video: distance
Video saved as es2_best_tracking_distance.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

1.14.5 Appendix: matching with temporal gap t → t+K

In this last additional section, we want to address the problem of matching the blobs across temporal gaps (for instance from t to t + 3, instead of adiacent frames). In order to do so, we are going to use the manual_static algorithm, with tau=0.1 and a gap equal to 3.

Let's define the functions the are useful to address this problem:

[64]:
```
GAP_K = 3
GAP_METHOD = 'manual_static'
GAP_MIN_AREA = 100
GAP_DETECT_KW = dict(tau=0.1, offset=0.1, k=1, alpha=0.1)
GAP_MATCH_KW = dict(w_dist=1, w_int=0, w_area=0, w_ecc=0, max_cost=0.25)
```

[65]:
```
def process_detection_mask(frame, bg, method, tau=0.1, offset=0.1, k=3.0,␣
↪alpha=0.1):
```

```
    """Same mapping as compute_blobs_with_tracking pass 1: (binary_mask,␣
↪updated_background)."""
    ALGO_REGISTRY = {
        "manual_static":  lambda f, b: (change_detection_with_tau(f, b,␣
↪tau)[0], b),
        "ksigma_static":  lambda f, b: (change_detection_ksigma(f, b, k=k)[0],␣
↪b),
        "mean_static":    lambda f, b: (change_detection_mean(f, b, offset)[0],␣
↪b),
        "otsu_static":    lambda f, b: (change_detection_otsu(f, b)[0], b),
        "manual_adaptive": lambda f, b: running_average_with_tau(f, b, tau,␣
↪alpha),
        "ksigma_adaptive": lambda f, b: running_average_ksigma(f, b, k=k,␣
↪alpha=alpha),
        "mean_adaptive":   lambda f, b: running_average_mean(f, b, offset,␣
↪alpha),
        "otsu_adaptive":   lambda f, b: running_average_otsu(f, b, alpha),
    }
    fn = ALGO_REGISTRY.get(method)
    if fn is None:
        raise ValueError(f"Method '{method}' not recognized.")
    return fn(frame, bg)


def label_props_at_index(frames, initial_bg, method, frame_idx, min_area=100,␣
↪**detect_kw):
    """Mask at frames[frame_idx] after evolving bg through frames 0..frame_idx␣
↪(matches pipeline)."""
    bg = initial_bg.copy()
    mask = None
    for i in range(frame_idx + 1):
        mask, bg = process_detection_mask(frames[i], bg, method, **detect_kw)
    cleaned = map_cleaning(mask)
    return compute_cc(cleaned, min_area=min_area)


def compute_gap_matching_visual_frames(
    frames,
    initial_bg,
    gap=3,
    method='manual_static',
    min_area=100,
    match_kwargs=None,
    **detect_kw,
):
    """Annotate frames[t+gap] with matches from t to t+gap."""
```

```
    if match_kwargs is None:
        match_kwargs = {}
    mk = {k: match_kwargs[k] for k in ('w_dist', 'w_int', 'w_area', 'w_ecc',␣
↪'max_cost') if k in match_kwargs}
    out_frames = []
    n = len(frames)
    for t in range(n - gap):
        _, props_t = label_props_at_index(frames, initial_bg, method, t,␣
↪min_area=min_area, **detect_kw)
        label_img_tg, props_tg = label_props_at_index(frames, initial_bg,␣
↪method, t + gap, min_area=min_area, **detect_kw)
        matches = match_blobs(props_t, props_tg, frames[t], frames[t + gap],␣
↪**mk)
        frame_vis = util.img_as_ubyte(frames[t + gap])
        if len(frame_vis.shape) == 2:
            frame_vis = cv2.cvtColor(frame_vis, cv2.COLOR_GRAY2RGB)
        overlay = frame_vis.copy()
        overlay[label_img_tg > 0] = (255, 255, 0)
        frame_vis = cv2.addWeighted(overlay, 0.3, frame_vis, 0.7, 0)
        for j, prop in enumerate(props_tg):
            minr, minc, maxr, maxc = prop.bbox
            cv2.rectangle(frame_vis, (minc, minr), (maxc, maxr), (0, 255, 255),␣
↪1)
            cv2.putText(frame_vis, f"B{j}", (minc, max(minr - 4, 8)), cv2.
↪FONT_HERSHEY_SIMPLEX, 0.35, (255, 255, 255), 1)
        for _i, j in matches:
            y1, x1 = props_t[int(_i)].centroid
            y2, x2 = props_tg[j].centroid
            p1, p2 = (int(x1), int(y1)), (int(x2), int(y2))
            cv2.circle(frame_vis, p2, 3, (0, 255, 0), -1)
            cv2.line(frame_vis, p1, p2, (255, 0, 255), 1)
        cv2.putText(
            frame_vis,
            f"{method} | gap={gap} | t={t}->{t+gap} | matches={len(matches)}",
            (10, 18),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.45,
            (255, 255, 255),
            1,
        )
        out_frames.append(frame_vis)
    return out_frames
```

At this point, let's compute the tracking on both videos:

[66]:
```
for video_name, vid_frames, vid_bg in (
    ('es1', es1_frames, bg_fixed),
```

```
    ('es2', tiny_split, bg_fixed_2nd),
):
    gap_frames = compute_gap_matching_visual_frames(
        vid_frames,
        vid_bg,
        gap=GAP_K,
        method=GAP_METHOD,
        min_area=GAP_MIN_AREA,
        match_kwargs=GAP_MATCH_KW,
        **GAP_DETECT_KW,
    )
    print(f"[{video_name}] gap={GAP_K} method={GAP_METHOD}: {len(gap_frames)}␣
↪annotated frames")
    export_and_show_video(gap_frames,␣
↪f"gap_matching_{video_name}_{GAP_METHOD}_K{GAP_K}.mp4")
```

```
[es1] gap=3 method=manual_static: 68 annotated frames
Video saved as gap_matching_es1_manual_static_K3.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

```
[es2] gap=3 method=manual_static: 265 annotated frames
Video saved as gap_matching_es2_manual_static_K3.mp4
Conversion successful! Displaying video…
```

```
<IPython.core.display.Video object>
```

It's pretty hard to make conclusions on these experiments, but what we can observe is the importance of the value of k that depends on the video that we are working on. In fact, $k = 3$ for the first video is a good choice since the people are slowly walking while for the second video the cars are moving quite fast and using $k = 3$ might result in worse results.